ANCHOR: KL-Anchored Adaptation for Stable Offline-to-Online RL

Geonwoo Cho[1] **Jaegyun Im**[2] **Doyoon Kim**[3] **Lexin Li**[4]
[1]University of Michigan, Ann Arbor
[2]Gwangju Institute of Science and Technology
[3]Pohang University of Science and Technology
[4]University of California, Berkeley
geonwoo@umich.edu

Abstract

Offline-to-online reinforcement learning (RL) follows the pretrain-finetune paradigm: an agent is first pretrained on historical data and then adapted through online interaction. We study offline-to-online adaptation without offline replay, where the agent must improve using only the pretrained model and newly collected experience. A key challenge is to avoid *catastrophic failure*, a sharp early performance collapse during online fine-tuning, while still achieving strong *asymptotic performance*. Through controlled experiments, we show that removing offline replay can induce catastrophic failure, while retaining strong conservative regularization from offline RL can suppress final online performance. We also find that common stabilization strategies do not reliably prevent early collapse, and that the critic-side mechanisms we test are insufficient on their own. Motivated by these findings, we propose ANCHOR, a simple adaptation method that anchors the actor to the offline-pretrained policy early in online fine-tuning, anneals this constraint over time, and retains only a small residual conservative critic regularizer. We provide a theoretical interpretation showing that KL anchoring helps preserve critic reliability across nearby occupancy distributions, with a two-phase regret bound explaining when anchoring should be maintained or relaxed. Across FrankaKitchen, AntMaze, and Adroit tasks, ANCHOR substantially reduces catastrophic failure while maintaining strong asymptotic performance. Ablations show that KL annealing and the residual conservative weight are important for balancing early stability and online adaptation, while additional analysis shows that ANCHOR is most beneficial when the pretrained policy contains useful behavior to preserve.

1 Introduction

The pretrain-finetune paradigm has driven much of the recent success in modern machine learning across diverse domains such as natural language processing and computer vision [Min et al., 2023, Khan et al., 2024]. Inspired by these advances, reinforcement learning (RL) has adopted a similar paradigm: an agent is first pretrained on historical offline data and then fine-tuned through online interaction with the target environment [Agarwal et al., 2022, Luo et al., 2024]. This setting, known as offline-to-online RL, allows the agent to effectively leverage prior knowledge while adapting to the target environment with fewer costly or risky online interactions.

Prior work in this direction has shown that modifying the training objective or algorithmic structure during the online phase leads to better performance than naively deploying offline RL algorithms without modification [Nakamoto et al., 2023, Zhou et al., 2025, Xiao et al., 2025]. However, catastrophic failure often occurs, i.e., there is an early collapse in performance when the agent first transitions from offline to online. This raises a central question: how can we adapt a pretrained agent

Preprint.

online while avoiding early collapse and maintaining strong asymptotic performance? This question is particularly critical in safety-sensitive applications such as healthcare and robotics [Singh et al., 2022, Liu et al., 2020]. In such settings, catastrophic failure is not merely a transient performance drop; it can lead to unsafe or harmful decisions during deployment.

We study this question on robot-control benchmarks, including FrankaKitchen and AntMaze from D4RL [Fu et al., 2020] and Adroit dexterous manipulation tasks [Nair et al., 2020]. Controlled experiments show that removing offline replay can induce catastrophic failure, while retaining the conservative regularization inherited from offline RL can reduce final online performance. We further find that common stabilization strategies, including UTD-ratio tuning [Xiao et al., 2025] and warmup-based online data collection [Zhou et al., 2025], do not reliably prevent early collapse. We also show that the critic-side mechanisms we test can be insufficient on their own.

Motivated by these findings, we propose **ANCHOR**, a simple KL-anchored adaptation method for offline-to-online fine-tuning without offline replay. ANCHOR regularizes the online actor toward the offline-pretrained policy on online replay states during the vulnerable early phase, and gradually removes this constraint through KL annealing to enable online improvement. For the critic, ANCHOR keeps only a small residual conservative regularization term, avoiding excessive pessimism while retaining mild protection against overestimation.

We provide theoretical and empirical support for this design. Our analysis interprets KL anchoring as a mechanism for preserving critic reliability across nearby occupancy distributions, and gives a two-phase regret bound explaining when anchoring should be maintained or relaxed depending on pretrained policy quality. Empirically, ANCHOR substantially reduces catastrophic failure while achieving strong asymptotic performance, with the best average final success rate across the six tasks. Ablations show that KL annealing improves over both keeping the KL constraint fixed and removing it from the start, that the residual conservative regularization controls the stability-adaptation balance, and that ANCHOR is most beneficial when the pretrained policy contains useful behavior to preserve.

2 Preliminaries

2.1 Conservative Q-Learning (CQL)

In offline RL, an agent is trained on a fixed dataset $\mathcal{D}$ collected by a behavior policy, without interacting with the environment. A key challenge is that standard Q-learning objectives can assign erroneously high values to actions not present in the dataset, leading to poor policy performance when deployed online. CQL [Kumar et al., 2020] addresses this issue by adding a regularization term to the standard temporal-difference (TD) loss, $\mathcal{L}_{\text{TD}}$, that penalizes Q-values of actions sampled from the policy relative to those from the dataset:

$$\mathcal{L}_{\text{CQL}} = \mathcal{L}_{\text{TD}} + \alpha \left(\mathbb{E}_{s\sim\mathcal{D},\, a\sim\pi(\cdot|s)}[Q_\theta(s,a)] - \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q_\theta(s,a)] \right), \tag{1}$$

where $\alpha > 0$ controls the penalty strength. This discourages overly optimistic Q-values for out-of-distribution actions. We adopt CQL as the backbone of our method.

2.2 Offline-to-Online Reinforcement Learning

Offline-to-online (O2O) RL considers the setting in which an agent is first pretrained on a fixed offline dataset and then further adapted through online interaction with the target environment [Nakamoto et al., 2023, Zhou et al., 2025, Xiao et al., 2025, Wen et al., 2024]. We focus on the setting where the original offline trajectories are not available during online adaptation. In this case, the agent must reuse knowledge encoded in the pretrained model while learning from an online data distribution.

This setting presents two simultaneous requirements: the agent should avoid early performance degradation during the transition, while still improving beyond the offline solution through online interaction. We evaluate these two aspects using two metrics. *Catastrophic failure* measures the drop from the initial online performance to the minimum success rate observed within the first 100K online steps; we use *early stability* to refer to low catastrophic failure. *Asymptotic performance* measures the average success rate over the final 50K online steps, corresponding to the 350K-400K window, and captures the late-stage plateau. Together, these metrics assess whether an O2O method can maintain early stability while enabling further online improvement. Further justification for these evaluation windows is provided in Appendix L.

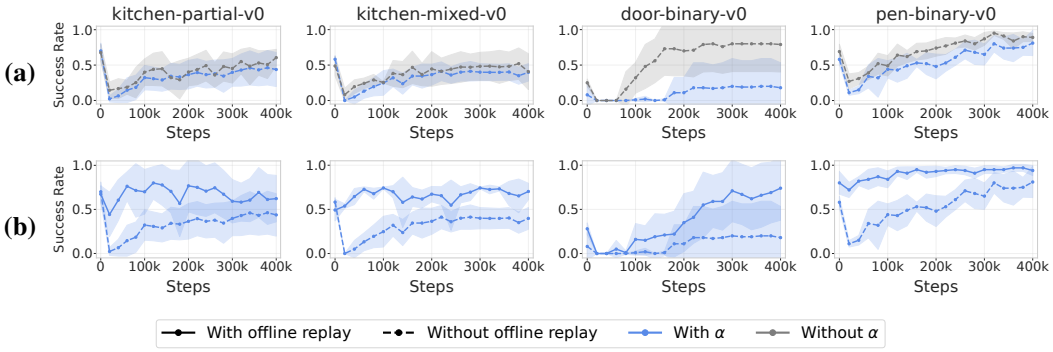


Figure 1: **Controlled analysis of offline data and conservative regularization.** Success rates of CQL during online fine-tuning across four tasks. (a) Keeping the conservative regularizer α during online fine-tuning can reduce asymptotic performance. (b) Removing offline replay can induce early performance degradation.

3 Diagnosing Failure Modes in Replay-Free Offline-to-Online RL

3.1 How Do Offline Data and Conservative Regularization Shape Online Adaptation?

We first examine the individual roles of offline data access and conservative regularization during the offline-to-online transition. To isolate their effects, we conduct controlled CQL experiments. Unless otherwise noted, all reported experiments are run with five random seeds, and we report the mean with 95% confidence intervals shown as shaded regions. Step 0 denotes the beginning of online fine-tuning. In Figure 1, "with offline replay" mixes offline data into online fine-tuning batches, while "without offline replay" uses only newly collected online data. Similarly, "with α" retains the CQL conservative regularizer during online fine-tuning, whereas "without α" removes it.

Our first observation is that conservative regularization can limit asymptotic online performance. To isolate the effect of the CQL regularizer, we remove offline replay in both settings and compare two variants: one that retains the conservative regularization coefficient α, and one that removes it. As shown in Figure 1(a), removing α achieves comparable or better final performance across all four tasks. This suggests that the pessimism induced by CQL, while useful for offline pretraining, can become restrictive once the policy must improve beyond the support of the offline dataset.

Our second observation is that removing offline data access can induce catastrophic early degradation. To isolate this effect, we compare standard CQL fine-tuning with and without offline replay. Figure 1(b) shows that removing offline replay often causes a sharp performance drop at the beginning of online fine-tuning. The main exception is door-binary, where the initial success rate after offline pretraining is already low, making the drop less pronounced. This indicates that offline replay serves as an important stabilizing mechanism during the early online phase.

Together, these results identify two distinct factors that shape offline-to-online adaptation: offline replay stabilizes the early online phase, while strong conservative regularization can restrict asymptotic improvement. When offline replay is unavailable, we therefore need an adaptation mechanism that prevents early catastrophic failure without maintaining overly conservative regularization throughout online fine-tuning.

3.2 Can Existing Stabilization Strategies Prevent Catastrophic Failure?

Section 3.1 shows that offline-to-online fine-tuning can become unstable without offline replay. We therefore test whether existing stabilization strategies can mitigate catastrophic failure in this setting.

We first show that standard stabilization strategies fail to consistently resolve early collapse across tasks. We consider the update-to-data (UTD) ratio, which increases the number of critic updates per online interaction and has been suggested to alleviate catastrophic failure [Xiao et al., 2025]. As shown in the top row of Figure 2, varying the UTD ratio does not consistently reduce early collapse across tasks. We next evaluate a warmup phase, where the agent collects online trajectories with the offline-pretrained policy before parameter updates, following Zhou et al. [2025]. As shown in the

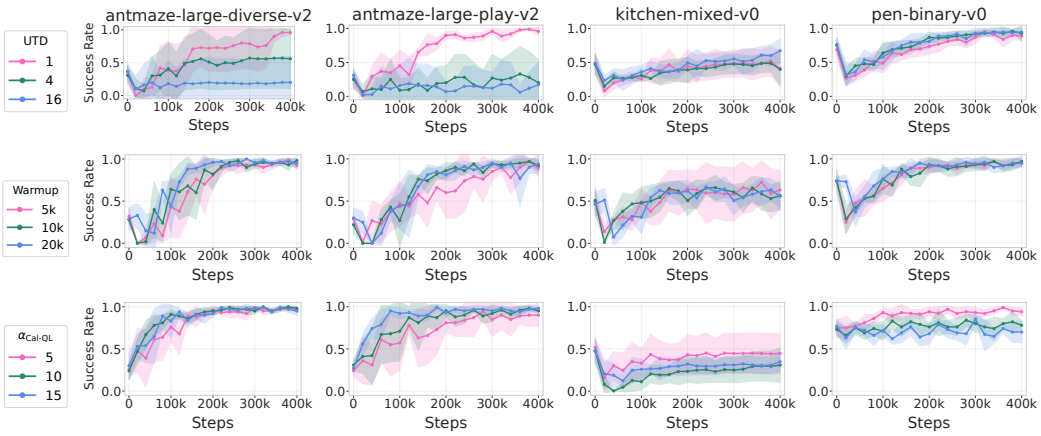


Figure 2: **Common stabilization strategies do not prevent catastrophic failure.** Effects of varying standard online fine-tuning choices across four tasks. *Top*: UTD ratio; *Middle*: warmup length; *Bottom*: Cal-QL conservative-regularizer weight, $\alpha_{\text{Cal-QL}}$. Increasing UTD, extending warmup, or tuning $\alpha_{\text{Cal-QL}}$ does not reliably prevent early performance collapse.



Figure 3: **Critic-shift regularization does not prevent catastrophic failure.** Effects of varying the critic-shift penalty strength α_{KL} across four tasks. *Top*: critic distribution shift during online fine-tuning, measured by $D_{\text{KL}}\big(\text{softmax}(Q_{\text{off}}) \,\|\, \text{softmax}(Q_\theta)\big)$. *Bottom*: corresponding success rates. Increasing α_{KL} slows the critic shift, but does not reliably prevent early performance collapse.

middle row of Figure 2, varying the warmup length also fails to reliably prevent catastrophic failure. Additional tuning attempts, including changes to batch size, learning rate, network width, hidden dimension, and LayerNorm, are reported in Appendix B. None of these modifications consistently prevent catastrophic failure across tasks.

We next test whether online calibration of the conservatively pretrained Q-function can stabilize fine-tuning. Cal-QL provides a natural testbed for this question, as it modifies the CQL conservative penalty to calibrate the Q-function using an online evidence-based reference value [Nakamoto et al., 2023]. The coefficient $\alpha_{\text{Cal-QL}}$ controls the strength of this calibrated conservative regularization. As shown in the bottom row of Figure 2, tuning $\alpha_{\text{Cal-QL}}$ can reduce early collapse in antmaze-large-diverse and antmaze-large-play, but the effect is not consistent across tasks or hyperparameter values. Moreover, Appendix L shows that ANCHOR achieves lower catastrophic failure than Cal-QL on average, suggesting that there remains room for improvement.

The limited effectiveness of Cal-QL raises the question of whether critic-side stabilization alone is sufficient. Motivated by the hypothesis of Zhou et al. [2025] that rapid critic shift contributes to early collapse, we test this question by explicitly penalizing deviation from the offline-pretrained critic:

$$\min_\theta \; \mathcal{L}_{\text{TD}}(\theta) + \alpha_{\text{KL}}\, \mathbb{E}_{(s,a)\sim\mathcal{B}} \left[D_{\text{KL}} \left(\text{softmax}\big(Q_{\text{offline}}(s,\cdot)\big) \,\|\, \text{softmax}\big(Q_{\text{online}}(s,\cdot)\big) \right) \right],$$

where $\mathcal{B}$ is the online replay buffer, D_{KL} denotes the KL divergence, Q_{offline} is the offline-pretrained critic, and Q_{online} is the critic being updated during online fine-tuning. The coefficient $\alpha_{\text{KL}} > 0$

controls the strength of this critic-shift penalty. As shown in Figure 3, increasing α_{KL} slows critic distribution shift, yet the corresponding success curves show that catastrophic failure persists. Appendix C further shows that Q-driven metrics do not reliably predict early collapse. These findings do not rule out critic instability as a contributing factor, but suggest that the critic-side stabilization strategies tested here are insufficient on their own, motivating policy-side stabilization.

4 Method

Building on Sections 3.1 and 3.2, we propose ANCHOR, a KL-anchored adaptation scheme that mitigates catastrophic failure while preserving high asymptotic performance during offline-to-online adaptation. We also provide a theoretical interpretation of ANCHOR.

4.1 ANCHOR: KL-anchored checkpoint adaptation

Let π_{off} denote the actor obtained after offline pretraining, and let π_t and Q_t denote the online actor and critic at online step t. ANCHOR regularizes the actor toward π_{off} during the early adaptation phase and gradually relaxes this constraint. Concretely, we use the actor objective

$$\mathcal{J}_{\pi,t}^{\mathrm{ANCHOR}}(\phi) = \mathbb{E}_{s\sim\mathcal{D}_{\mathrm{on}},\, a\sim\pi_\phi(\cdot|s)}\left[Q_t(s,a)\right] - \beta_t\, \mathbb{E}_{s\sim\mathcal{D}_{\mathrm{on}}}\left[D_{\mathrm{KL}}\big(\pi_\phi(\cdot|s)\,\|\,\pi_{\mathrm{off}}(\cdot|s)\big)\right], \tag{2}$$

where $\beta_t \geq 0$ is the KL regularization weight.

The critic is updated with the usual online TD loss together with a conservative CQL term whose coefficient is fixed to α_{on} during online fine-tuning:

$$\mathcal{L}_Q^{\mathrm{ANCHOR}}(\theta) = \mathcal{L}_{\mathrm{TD}}(\theta;\mathcal{D}_{\mathrm{on}}) + \alpha_{\mathrm{on}}\Big(\mathbb{E}_{s\sim\mathcal{D}_{\mathrm{on}},\, a\sim\pi_t(\cdot|s)}[Q_\theta(s,a)] - \mathbb{E}_{(s,a)\sim\mathcal{D}_{\mathrm{on}}}[Q_\theta(s,a)]\Big). \tag{3}$$

Here, $\alpha_{\mathrm{on}} \geq 0$ denotes the residual conservative regularization strength used throughout online fine-tuning, which is set smaller than the coefficient used during offline pretraining. This design avoids excessive conservatism that may limit online improvement, while retaining a small amount of critic regularization to prevent abrupt critic distribution shift and stabilize adaptation.

For intuition, consider a fixed state s and an unconstrained policy class over a continuous action space $\mathcal{A}$. Assuming policies are absolutely continuous with respect to π_{off} and the normalizer is finite, the state-wise objective in Equation 2 admits the closed-form solution [Foster et al., 2025]

$$\pi_t^\star(a|s) = \frac{\pi_{\mathrm{off}}(a|s)\exp(Q_t(s,a)/\beta_t)}{Z_t(s)}, \qquad Z_t(s) = \int_{\mathcal{A}} \pi_{\mathrm{off}}(a'|s)\exp(Q_t(s,a')/\beta_t)\; da'. \tag{4}$$

Thus, β_t controls the strength of anchoring: as $\beta_t \to 0$, the ideal policy concentrates on high-Q_t actions, while large β_t keeps it close to π_{off}. In our continuous-control experiments, we do not compute this nonparametric solution explicitly; instead, we optimize the actor using Equation 2.

In practice, ANCHOR uses a linear KL annealing schedule:

$$\beta_t = \beta_0 \cdot \max\left(1 - \frac{t}{T_{\mathrm{KL}}},\, 0\right),$$

where β_0 is the initial KL regularization weight and T_{KL} is the KL annealing interval. Thus, β_t decreases linearly from β_0 to 0 during the first T_{KL} online steps. After $t \geq T_{\mathrm{KL}}$, the KL anchoring term is completely removed and the actor is optimized using the standard online objective. Exact hyperparameter settings are reported in Appendix I. We detail the difference from trust-region methods such as TRPO [Schulman et al., 2015] in Appendix G.

4.2 Theoretical Interpretation

We provide a formal interpretation of ANCHOR through two supporting lemmas. The first lemma shows that KL anchoring can control the distribution-shift component of critic instability. The second lemma gives a two-phase regret bound that quantifies the cost of using KL anchoring during the early online phase before switching to the unanchored online learner. Detailed assumptions and proofs are deferred to Appendix A.

Lemma 1 (Critic error difference under KL-occupancy control)**.** *Let $\widehat{Q}_t$ denote the learned critic at online step t, and let Q^π denote the true action-value function of an arbitrary target policy π. Define*

$$e_t(s,a) = |\widehat{Q}_t(s,a) - Q^\pi(s,a)|.$$

If $|\widehat{Q}_t(s,a)| \le Q_{\max}$ and $|Q^\pi(s,a)| \le Q_{\max}$, then for any two occupancy measures d and d',

$$|\mathbb{E}_d[e_t] - \mathbb{E}_{d'}[e_t]| \le 2Q_{\max}\|d - d'\|_1.$$

If the online actor additionally satisfies a KL constraint around the offline policy,

$$D_{d^{\pi_{\text{off}}}}(\pi_t \| \pi_{\text{off}}) \le \delta_t,$$

and the occupancy shift is locally controlled by policy divergence, then

$$|\mathbb{E}_{d^{\pi_t}}[e_t] - \mathbb{E}_{d^{\pi_{\text{off}}}}[e_t]| = O(\sqrt{\delta_t}).$$

Lemma 1 is a distribution-transfer bound for critic error. It does not assume that the critic error is small; instead, it shows that if the critic is accurate under the offline-policy occupancy, then its error under the current online-policy occupancy remains controlled when π_t stays close to π_{off}. Thus, if the pretrained critic is reliable on the offline data distribution and this reliability is preserved during early online updates, KL anchoring prevents the online policy from moving too quickly into regions where critic error may be much larger.

In practice, ANCHOR computes the KL penalty on online replay states rather than under the offline-policy occupancy used in the lemma. Although these distributions differ, the replay-buffer KL serves the same local role: it constrains policy updates on states encountered during fine-tuning and thereby limits abrupt policy drift.

Lemma 2 (Two-phase regret decomposition)**.** *Over $T = T_1 + T_2$ steps, suppose phase I uses KL anchoring around π_{off} with budgets δ_t. In phase II, suppose the actor-side KL anchor is removed, and the resulting online learner has regret $\mathrm{Reg}_{\text{base}}(T_2)$. Then under bounded advantage and local occupancy control,*

$$\mathrm{Reg}_T \le T_1\Delta_{\text{off}} + C\sum_{t=1}^{T_1}\sqrt{\delta_t} + \mathrm{Reg}_{\text{base}}(T_2),$$

where $\Delta_{\text{off}} = J(\pi^*) - J(\pi_{\text{off}})$ and C is a problem-dependent constant.

Lemma 2 separates the regret into the cost of staying near the offline policy, the cost of controlled policy deviation during the anchored phase, and the regret after the anchor is removed. This decomposition clarifies how the usefulness of KL anchoring depends on the quality of the pretrained policy. When π_{off} is weak, Δ_{off} is large, so remaining close to the offline policy for too long can incur substantial regret; in this case, annealing the KL coefficient is important for allowing the actor to move beyond the pretrained solution. When π_{off} is strong, Δ_{off} is small, so the cost of staying near the offline policy is limited, and stronger or longer anchoring can be justified if it prevents unstable online updates.

Overall, the analysis supports the central design of ANCHOR: KL anchoring limits early distribution shift and helps preserve critic reliability near the pretrained policy, while annealing removes this constraint when online improvement beyond the pretrained solution is needed. The results should be viewed as an interpretation of the stabilizing role of anchoring, rather than a convergence guarantee.

5 Experiments

5.1 Experimental Setup

Environments and tasks. We evaluate ANCHOR on three widely used offline-to-online RL benchmarks: FrankaKitchen and AntMaze from D4RL [Fu et al., 2020], and Adroit dexterous manipulation tasks [Nair et al., 2020]. Our main benchmark consists of six tasks: kitchen-mixed, kitchen-partial, antmaze-large-diverse, antmaze-large-play, door-binary, and pen-binary. All offline pretraining datasets match those used in WSRL [Zhou et al., 2025]. Further environment details and task visualizations are provided in Appendices I and K.

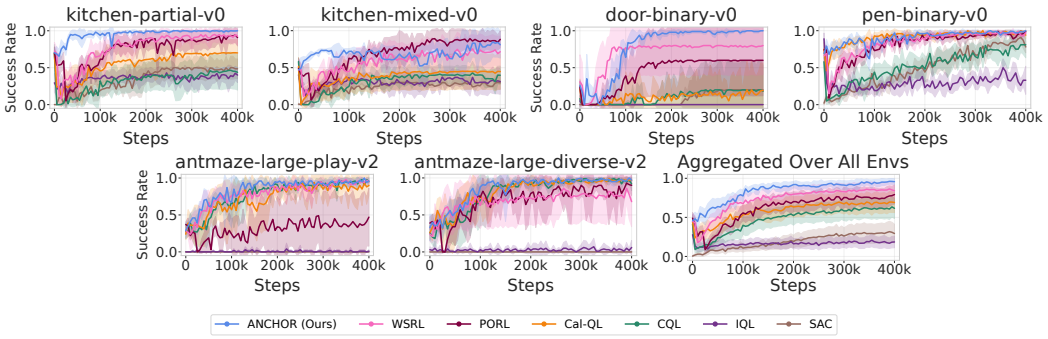


Figure 4: **ANCHOR improves both early stability and final performance.** Existing methods do not consistently achieve both low catastrophic failure and strong asymptotic performance across tasks. In contrast, ANCHOR reduces early performance collapse while attaining strong final performance across tasks. The aggregate curve reports the mean success rate averaged over the six tasks.

Training procedure. We first perform offline pretraining for 1M steps in AntMaze, 250K steps in FrankaKitchen, and 40K steps in Adroit, followed by 400K steps of online fine-tuning. All methods are trained without offline data replay and use the same online interaction budget.

Baselines. We compare ANCHOR against WSRL [Zhou et al., 2025], PORL [Xiao et al., 2025], Cal-QL [Nakamoto et al., 2023], CQL [Kumar et al., 2020], IQL [Kostrikov et al., 2022], and SAC [Haarnoja et al., 2018]. Detailed descriptions of the baselines are provided in Appendix H.

For baseline reproduction, we largely follow the experimental setup of WSRL [Zhou et al., 2025]. Any modifications made to stabilize training are described in Appendix I.3. ANCHOR is implemented on top of CQL, with modifications to the online training objective. Details of the ANCHOR implementation and hyperparameters are provided in Appendix I.

5.2 ANCHOR Improves Both Early Stability and Final Performance

Figure 4 summarizes the main benchmark results. Across the six-task suite, ANCHOR achieves the best average final performance while substantially reducing catastrophic failure. In door-binary, where the offline-pretrained policy starts from relatively low performance, the benefit of preventing catastrophic failure is less pronounced; nevertheless, ANCHOR achieves the best asymptotic performance.

Existing baselines often fail to achieve both early stability and strong final performance. Cal-QL, which retains stronger conservative regularization, can reduce early collapse but often adapts more slowly, whereas more aggressive online adaptation methods, including WSRL and PORL, can suffer larger early performance drops. Full numerical results, reported as mean $\pm$ standard error, are provided in Appendix L.

We further evaluate ANCHOR in additional environments that stress different failure modes: near-zero pretrained performance in kitchen-complete and relocate-binary, and harder sparse-reward exploration in antmaze-ultra-diverse (Appendix F). In kitchen-complete, ANCHOR remains competitive with the KL coefficient set to zero, but, like other baselines, fails to improve asymptotic performance in relocate-binary. In antmaze-ultra-diverse, ANCHOR also fails to improve over the pretrained policy. These results suggest that KL anchoring is useful when there is pretrained behavior to preserve, but may need to be combined with stronger exploration or adaptation mechanisms in harder failure cases.

We also analyze whether KL-anchored adaptation can be integrated with another offline RL backbone, IQL. Appendix E shows that adding KL anchoring to IQL reduces early catastrophic failure compared to naive online fine-tuning, although it does not improve asymptotic performance.

5.3 Analysis of ANCHOR

Annealing the KL constraint balances early anchoring and later adaptation. We first ask whether the KL constraint should be kept throughout online fine-tuning, removed from the beginning, or annealed over time. To answer this, we compare ANCHOR against two alternatives: fixed-KL, which

keeps the initial KL weight throughout online fine-tuning without annealing, and no-KL, which does not use KL regularization.

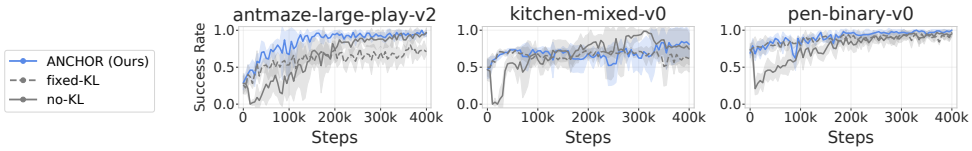


Figure 5: **KL annealing improves early stability while preserving final performance.** ANCHOR is compared with the fixed-KL and no-KL variants. The fixed-KL variant improves early stability but can limit final improvement, while the no-KL variant adapts aggressively but causes early collapse.

As shown in Figure 5, ANCHOR most consistently achieves both low catastrophic failure and strong final performance across the three tasks. The fixed-KL variant demonstrates the benefit of anchoring: keeping the actor close to the pretrained policy can suppress early collapse. However, because this constraint is never annealed, it can remain overly restrictive and reduce asymptotic performance, especially when the offline-pretrained policy is not already strong, as in antmaze-large-play and kitchen-mixed. Conversely, the no-KL variant removes this restriction entirely and allows more aggressive online adaptation, but it exhibits large catastrophic failure across all three tasks because the policy can drift away from the pretrained behavior too abruptly. These patterns justify the annealing design of ANCHOR: KL annealing provides a middle path by using strong anchoring when online fine-tuning is most vulnerable, and then relaxing the constraint as the online policy and critic adapt.

The benefit of ANCHOR depends on the quality of the pretrained policy. We next examine when ANCHOR is most beneficial by varying the quality of the offline-pretrained policy. We compare ANCHOR with the no-KL variant across pretrained checkpoints with different initial success rates.

Figure 6 shows that the effect of ANCHOR depends on the quality of the offline-pretrained policy. In both pen-binary and kitchen-mixed, when the pretrained policy starts with a low initial success rate, the difference in catastrophic failure between ANCHOR and the no-KL variant is less pronounced. In contrast, when the pretrained policy starts from a stronger initial success rate, the gap becomes much clearer: ANCHOR substantially reduces catastrophic failure relative to the no-KL variant.

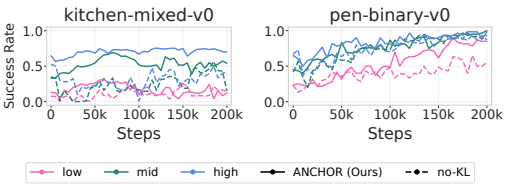


Figure 6: **Benefit of ANCHOR depends on pretrained policy quality.** ANCHOR reduces catastrophic failure more clearly when the offline-pretrained policy starts from a higher initial success rate. Curves with the same color correspond to the same pretrained policy. Results are averaged over three seeds.

Importantly, even in the low-initial-success regime, ANCHOR remains a reasonable design choice. Compared to the no-KL variant, it still achieves strong asymptotic performance in pen-binary and comparable asymptotic performance in kitchen-mixed. Taken together, these results indicate that the main value of ANCHOR lies in protecting useful pretrained behavior when it exists, while not substantially hindering final online performance even when the pretrained policy is less reliable. Results with full confidence intervals are provided in Appendix L.

How sensitive is ANCHOR to its hyperparameters? We finally study the sensitivity of ANCHOR to three hyperparameters: the initial KL weight β_0, the KL annealing interval T_{KL}, and the residual online conservative regularization strength α_{on}.

Figure 7 shows that ANCHOR is robust to moderate changes in its KL-related hyperparameters. Although different choices of β_0 and T_{KL} can affect learning stability, all tested variants continue to reduce catastrophic failure across the three tasks and achieve similar asymptotic performance within confidence intervals. Thus, ANCHOR's gains are not driven by a single carefully tuned configuration, but reflect a robust effect of KL-anchored adaptation.

The residual conservative weight α_{on} controls a different aspect of the method. As shown in Figure 7, we compare three choices: removing the conservative penalty, using an intermediate residual value between zero and the offline-pretraining weight, and retaining the full offline-pretraining

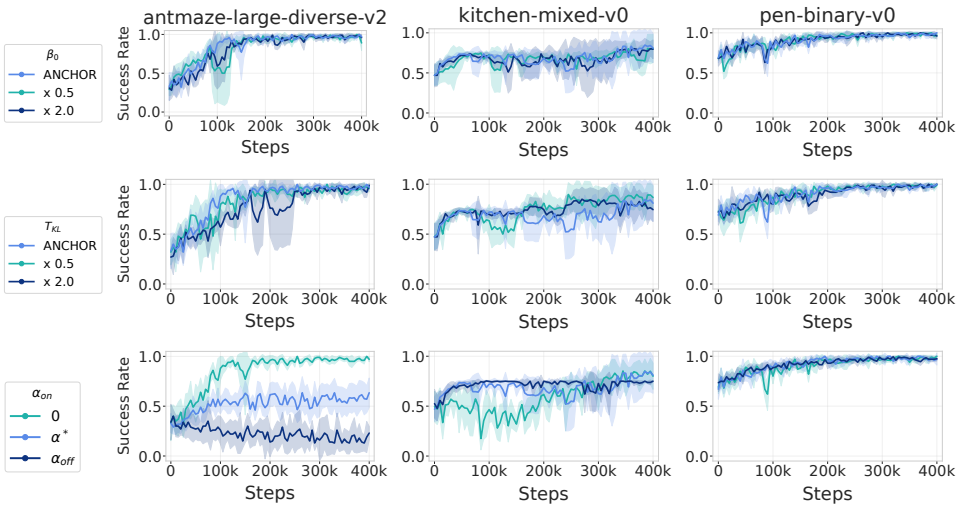


Figure 7: **Hyperparameter sensitivity of ANCHOR.** We vary the initial KL weight β_0 (top), the KL annealing interval T_{KL} (middle), and the residual online conservative weight α_{on} (bottom). Here, α^* denotes an intermediate value between 0 and α_{off}, where α_{off} is the conservative regularization weight used during offline pretraining. ANCHOR remains stable under moderate changes to KL-related hyperparameters, while α_{on} controls the balance between early stability and final improvement.

weight. Retaining the full offline conservative weight leads to the lowest asymptotic performance in antmaze-large-diverse and kitchen-mixed, consistent with the analysis in Section 3. Removing the conservative penalty entirely achieves strong final performance across tasks, but causes larger catastrophic failure in kitchen-mixed. These results support the design choice of ANCHOR: reducing conservative regularization can improve online adaptability, while retaining a small residual penalty can help stabilize the early online phase depending on the task. Additional ablation studies on other hyperparameters and an analysis of how the initial KL weight affects policy distribution shift are provided in Appendix D.

6 Conclusion

Offline-to-online adaptation without offline replay requires balancing early stability with online improvement. ANCHOR addresses this by anchoring the actor to the offline-pretrained policy during the vulnerable early phase, annealing this constraint as online learning progresses, and retaining only a small residual conservative penalty for the critic. Across FrankaKitchen, AntMaze, and Adroit tasks, this simple design reduces catastrophic failure while maintaining strong asymptotic performance.

Our analysis and ablations clarify the operating conditions of ANCHOR. The theoretical results suggest that anchoring is most useful when the pretrained policy induces nearby occupancy distributions under which the critic remains reliable, while the two-phase regret bound explains why the anchor should eventually be relaxed when further online improvement is needed. Empirically, the ablations show that ANCHOR is not simply a fixed constraint: its benefit comes from using the pretrained policy as a temporary behavioral prior, while allowing both the actor and critic to adapt as online data accumulates. They also show that the residual conservative penalty should be kept small enough to avoid limiting final performance, but can still help stabilize the early online phase in some tasks.

These findings suggest that offline-to-online RL methods should control how strongly and how long they rely on the pretrained policy during deployment-time adaptation. Future work could develop adaptive anchoring schedules that adjust the KL constraint based on online signals. Another important direction is to better disentangle the roles of actor drift and critic error in catastrophic failure, which may lead to more targeted stabilization mechanisms. Finally, settings where the pretrained policy provides little useful guidance may require combining anchoring with stronger exploration mechanisms that enable online improvement without reintroducing early collapse. Additional limitations are discussed in Appendix J.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Reincarnating reinforcement learning: Reusing prior computation to accelerate progress. *Advances in Neural Information Processing Systems*, 35:28955–28971, 2022.

Gaon An, Seungyong Moon, Jang-Hyun Kim, and Hyun Oh Song. Uncertainty-based offline reinforcement learning with diversified q-ensemble. *Advances in Neural Information Processing Systems*, 34:7436–7447, 2021.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 1577–1594. PMLR, 2023.

David Brandfonbrener, Will Whitney, Rajesh Ranganath, and Joan Bruna. Offline rl without off-policy evaluation. *Advances in Neural Information Processing Systems*, 34:4933–4946, 2021.

Xiaocong Chen, Siyu Wang, Julian McAuley, Dietmar Jannach, and Lina Yao. On the opportunities and challenges of offline reinforcement learning for recommender systems. *ACM Transactions on Information Systems*, 42(6):1–26, 2024.

Xinyue Chen, Che Wang, Zijian Zhou, and Keith Ross. Randomized ensembled double q-learning: Learning fast without a model. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. arXiv preprint arXiv:2101.05982.

Rohan Chitnis, Yingchen Xu, Bobak Hashemi, Lucas Lehnert, Urun Dogan, Zheqing Zhu, and Olivier Delalleau. Iql-td-mpc: Implicit q-learning for hierarchical model predictive control. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9154–9160. IEEE, 2024.

Benjamin Eysenbach, Tianjun Zhang, Sergey Levine, and Russ R Salakhutdinov. Contrastive learning as goal-conditioned reinforcement learning. *Advances in Neural Information Processing Systems*, 35:35603–35620, 2022.

Jiaheng Feng, Mingxiao Feng, Haolin Song, Wengang Zhou, and Houqiang Li. Suf: Stabilized unconstrained fine-tuning for offline-to-online reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 11961–11969, 2024.

Dylan J. Foster, Zakaria Mhammedi, and Dhruv Rohatgi. Is a good foundation necessary for efficient reinforcement learning? the computational role of the base model in exploration. In *Proceedings of the Thirty Eighth Conference on Learning Theory*, Proceedings of Machine Learning Research. PMLR, 2025.

Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.

Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. *Advances in Neural Information Processing Systems*, 34:20132–20145, 2021.

Divyansh Garg, Joey Hejna, Matthieu Geist, and Stefano Ermon. Extreme q-learning: Maxent rl without entropy. *arXiv preprint arXiv:2301.02328*, 2023.

Siyuan Guo, Yanchao Sun, Jifeng Hu, Sili Huang, Hechang Chen, Haiyin Piao, Lichao Sun, and Yi Chang. A simple unified uncertainty-guided framework for offline-to-online reinforcement learning. *arXiv preprint arXiv:2306.07541*, 2023.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 1861–1870. PMLR, 2018.

Hao Hu, Yiqin Yang, Jianing Ye, Chengjie Wu, Ziqing Mai, Yujing Hu, Tangjie Lv, Changjie Fan, Qianchuan Zhao, and Chongjie Zhang. Bayesian design principles for offline-to-online reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, ICML'24. JMLR.org, 2024.

Hengyuan Hu, Suvir Mirchandani, and Dorsa Sadigh. Imitation bootstrapped reinforcement learning. *arXiv preprint arXiv:2311.02198*, 2023.

Asifullah Khan, Anabia Sohail, Mustansar Fiaz, Mehdi Hassan, Tariq Habib Afridi, Sibghat Ullah Marwat, Farzeen Munir, Safdar Ali, Hannan Naseem, Muhammad Zaigham Zaheer, et al. A survey of the self supervised learning mechanisms for vision transformers. *arXiv preprint arXiv:2408.17059*, 2024.

Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. Morel: Model-based offline reinforcement learning. *Advances in Neural Information Processing Systems*, 33: 21810–21823, 2020.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Ilya Kostrikov, Rob Fergus, Jonathan Tompson, and Ofir Nachum. Offline reinforcement learning with fisher divergence critic regularization. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 5774–5783. PMLR, 2021.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 1179–1191, 2020.

Seunghyun Lee, Younggyo Seo, Kimin Lee, Pieter Abbeel, and Jinwoo Shin. Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble. In *Conference on Robot Learning*, pages 1702–1712. PMLR, 2022.

Siqi Liu, Kay Choong See, Kee Yuan Ngiam, Leo Anthony Celi, Xudong Sun, and Mengling Feng. Reinforcement learning for clinical decision support in critical care: Comprehensive review. *Journal of Medical Internet Research*, 22(7):e18477, 2020. doi: 10.2196/18477.

Qin-Wen Luo, Ming-Kun Xie, Yewen Wang, and Sheng-Jun Huang. Optimistic critic reconstruction and constrained fine-tuning for general offline-to-online rl. *Advances in Neural Information Processing Systems*, 37:108167–108207, 2024.

Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 56(2):1–40, 2023.

Ashvin Nair, Abhishek Gupta, Murtaza Dalal, and Sergey Levine. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.

Mitsuhiko Nakamoto, Simon Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. In *Advances in Neural Information Processing Systems*, volume 36, pages 62244–62269, 2023.

Whitney K Newey and James L Powell. Asymmetric least squares estimation and testing. *Econometrica: Journal of the Econometric Society*, pages 819–847, 1987.

Alexander Nikulin, Vladislav Kurenkov, Denis Tarasov, and Sergey Kolesnikov. Anti-exploration by random network distillation. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 26228–26244. PMLR, 2023.

Kwanyoung Park and Youngwoon Lee. Model-based offline reinforcement learning with lower expectile q-learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

Seohong Park, Qiyang Li, and Sergey Levine. Flow q-learning. *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.

Rafael Figueiredo Prudencio, Marcos ROA Maximo, and Esther Luna Colombini. A survey on offline reinforcement learning: Taxonomy, review, and open problems. *IEEE Transactions on Neural Networks and Learning Systems*, 35(8):10237–10257, 2023.

Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087*, 2017.

John Schulman, Sergey Levine, Pieter Abbeel, Michael I. Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 2015.

Yongjae Shin, Jeonghye Kim, Whiyoung Jung, Sunghoon Hong, Deunsol Yoon, Youngsoo Jang, Geonhyeong Kim, Jongseong Chae, Youngchul Sung, Kanghoon Lee, et al. Online pre-training for offline-to-online reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. URL `https://openreview.net/forum?id=5anr1H5vR5`.

Bharat Singh, Rajesh Kumar, and Vinay Pratap Singh. Reinforcement learning in robotic applications: A comprehensive survey. *Artificial Intelligence Review*, 55(2):945–990, 2022. doi: 10.1007/s10462-021-10067-x.

Denis Tarasov, Vladislav Kurenkov, Alexander Nikulin, and Sergey Kolesnikov. Revisiting the minimalist approach to offline reinforcement learning. *Advances in Neural Information Processing Systems*, 36:11592–11620, 2023.

Ikechukwu Uchendu, Ted Xiao, Yao Lu, Banghua Zhu, Mengyuan Yan, Joséphine Simon, Matthew Bennice, Chuyuan Fu, Cong Ma, Jiantao Jiao, et al. Jump-start reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 34556–34583. PMLR, 2023.

Ziyu Wang, Alexander Novikov, Konrad Zolna, Josh S Merel, Jost Tobias Springenberg, Scott E Reed, Bobak Shahriari, Noah Siegel, Caglar Gulcehre, Nicolas Heess, et al. Critic regularized regression. *Advances in Neural Information Processing Systems*, 33:7768–7778, 2020.

Xiaoyu Wen, Xudong Yu, Rui Yang, Haoyuan Chen, Chenjia Bai, and Zhen Wang. Towards Robust Offline-to-Online Reinforcement Learning via Uncertainty and Smoothness. *Journal of Artificial Intelligence Research*, 81:481–509, 2024. doi: 10.1613/jair.1.16457. URL `https://jair.org/index.php/jair/article/view/16457`.

Wei Xiao, Jiacheng Liu, Zifeng Zhuang, Runze Suo, Shangke Lyu, and Donglin Wang. Efficient online rl fine tuning with offline pre-trained policy only. *arXiv preprint arXiv:2505.16856*, 2025.

Haoran Xu, Li Jiang, Jianxiong Li, Zhuoran Yang, Zhaoran Wang, Victor Wai Kin Chan, and Xianyuan Zhan. Offline rl with no ood actions: In-sample learning via implicit value regularization. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Y Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization. *Advances in Neural Information Processing Systems*, 33:14129–14142, 2020.

Tianhe Yu, Aviral Kumar, Rafael Rafailov, Aravind Rajeswaran, Sergey Levine, and Chelsea Finn. Combo: Conservative offline model-based policy optimization. *Advances in Neural Information Processing Systems*, 34:28954–28967, 2021.

Zishun Yu and Xinhua Zhang. Actor-Critic Alignment for Offline-to-Online Reinforcement Learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 40452–40474, 2023.

Kai Zhao, Jianye Hao, Yi Ma, Jinyi Liu, Yan Zheng, and Zhaopeng Meng. Enoto: Improving offline-to-online reinforcement learning with q-ensembles. page 2609–2611, 2023.

Zhiyuan Zhou, Andy Peng, Qiyang Li, Sergey Levine, and Aviral Kumar. Efficient online reinforcement learning fine-tuning need not retain offline data. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

A Theoretical Discussion

This appendix provides a formal discussion of ANCHOR in the offline-to-online setting. For simplicity, we present the formal argument for finite action spaces. The same interpretation extends to continuous actions by replacing sums with integrals under standard absolute-continuity and boundedness assumptions. Our goal is to formalize the local occupancy-control intuition rather than to provide a full convergence guarantee for the continuous-control implementation.

Notation. We consider an infinite-horizon discounted MDP with discount factor $\gamma \in (0,1)$, finite action set $\mathcal{A}$, reward function $r(s,a)$, and normalized discounted state-action occupancy

$$d^\pi(s,a) := (1-\gamma)\sum_{k=0}^{\infty}\gamma^k \Pr_\pi(s_k = s, a_k = a).$$

With this normalization, d^π is a probability distribution over (s,a) and

$$J(\pi) = \frac{1}{1-\gamma}\,\mathbb{E}_{(s,a)\sim d^\pi}[r(s,a)].$$

For two probability distributions P and Q over a finite set, we define

$$D_{\mathrm{KL}}(P\|Q) := \sum_x P(x)\log\frac{P(x)}{Q(x)}, \qquad \mathrm{TV}(P,Q) := \frac{1}{2}\sum_x |P(x)-Q(x)| = \frac{1}{2}\|P-Q\|_1.$$

Under this convention, $\mathrm{TV}(P,Q) \in [0,1]$ and Pinsker's inequality gives

$$\mathrm{TV}(P,Q) \le \sqrt{\frac{1}{2}D_{\mathrm{KL}}(P\|Q)}.$$

For two policies π, μ and a reference state distribution ρ, define the state-averaged KL and TV distances as

$$D_\rho(\pi\|\mu) := \mathbb{E}_{s\sim\rho}[D_{\mathrm{KL}}(\pi(\cdot|s)\|\mu(\cdot|s))], \qquad \mathrm{TV}_\rho(\pi,\mu) := \mathbb{E}_{s\sim\rho}[\mathrm{TV}(\pi(\cdot|s),\mu(\cdot|s))].$$

We also write

$$A^\mu(s,a) := Q^\mu(s,a) - V^\mu(s).$$

Performance difference lemma. We will repeatedly use the standard identity

$$J(\pi) - J(\mu) = \frac{1}{1-\gamma}\,\mathbb{E}_{(s,a)\sim d^\pi}[A^\mu(s,a)]. \tag{5}$$

This is the classical performance difference lemma.

Assumption 1 (Local occupancy control around the offline policy)**.** *There exists a constant $C_{\mathrm{occ}} > 0$ such that, for every policy π in the anchored phase,*

$$\|d^\pi - d^{\pi_{\mathrm{off}}}\|_1 \le C_{\mathrm{occ}}\,\mathrm{TV}_{d^{\pi_{\mathrm{off}}}}(\pi,\pi_{\mathrm{off}}).$$

Assumption 1 is a local smoothness condition on the policy-to-occupancy map around π_{off}: within the anchored region, small policy deviations lead to proportionally controlled occupancy shifts. It excludes pathological cases in which a change on states with negligible offline visitation produces a large downstream change in the trajectory distribution. Since ANCHOR applies KL anchoring precisely during the early phase to keep the policy close to π_{off} on sampled states, our lemma is intended to characterize this local anchored regime rather than arbitrary policies far from the offline reference.

A.1 Lemma 1: Critic error under occupancy shift

Lemma 1 (Critic error difference under occupancy shift)**.** *Let $\widehat{Q}_t$ be a critic trained on online data collected from occupancy d_t. Fix any target policy π, and define*

$$e_t(s,a) := \left|\widehat{Q}_t(s,a) - Q^\pi(s,a)\right|.$$

Assume that

$$|\widehat{Q}_t(s,a)| \le Q_{\max}, \qquad |Q^\pi(s,a)| \le Q_{\max} \qquad \textit{for all } (s,a).$$

Then, for any two occupancy measures d and d',

$$\left|\mathbb{E}_{(s,a)\sim d}[e_t(s,a)] - \mathbb{E}_{(s,a)\sim d'}[e_t(s,a)]\right| \le 2Q_{\max}\|d - d'\|_1. \tag{6}$$

In particular, if the online actor satisfies

$$D_{d^{\pi_{\text{off}}}}(\pi_t \| \pi_{\text{off}}) \le \delta_t,$$

then under Assumption 1,

$$\|d^{\pi_t} - d^{\pi_{\text{off}}}\|_1 \le C_{\text{occ}}\sqrt{\delta_t/2}, \tag{7}$$

and therefore

$$|\mathbb{E}_{d^{\pi_t}}[e_t] - \mathbb{E}_{d^{\pi_{\text{off}}}}[e_t]| \le 2Q_{\max}C_{\text{occ}}\sqrt{\delta_t/2}. \tag{8}$$

Proof. By the boundedness assumptions and the triangle inequality,

$$e_t(s,a) = |\widehat{Q}_t(s,a) - Q^\pi(s,a)| \le |\widehat{Q}_t(s,a)| + |Q^\pi(s,a)| \le 2Q_{\max}.$$

Thus, for any two occupancy measures d and d',

$$\mathbb{E}_d[e_t] - \mathbb{E}_{d'}[e_t] = \sum_{s,a} \big(d(s,a) - d'(s,a)\big)e_t(s,a).$$

Taking absolute values and applying Hölder's inequality,

$$|\mathbb{E}_d[e_t] - \mathbb{E}_{d'}[e_t]| \le \|e_t\|_\infty \|d - d'\|_1 \le 2Q_{\max}\|d - d'\|_1.$$

This proves (6).

Now suppose

$$D_{d^{\pi_{\text{off}}}}(\pi_t \| \pi_{\text{off}}) \le \delta_t.$$

By Pinsker's inequality, for each state s,

$$\text{TV}(\pi_t(\cdot|s), \pi_{\text{off}}(\cdot|s)) \le \sqrt{\frac{1}{2}D_{\text{KL}}(\pi_t(\cdot|s)\|\pi_{\text{off}}(\cdot|s))}.$$

Taking expectation over $s \sim d^{\pi_{\text{off}}}$ and applying Jensen's inequality,

$$\text{TV}_{d^{\pi_{\text{off}}}}(\pi_t, \pi_{\text{off}}) \le \sqrt{\frac{1}{2}D_{d^{\pi_{\text{off}}}}(\pi_t\|\pi_{\text{off}})} \le \sqrt{\delta_t/2}.$$

Combining this with Assumption 1 gives

$$\|d^{\pi_t} - d^{\pi_{\text{off}}}\|_1 \le C_{\text{occ}}\sqrt{\delta_t/2}.$$

Finally, applying (6) with $d = d^{\pi_t}$ and $d' = d^{\pi_{\text{off}}}$ yields

$$|\mathbb{E}_{d^{\pi_t}}[e_t] - \mathbb{E}_{d^{\pi_{\text{off}}}}[e_t]| \le 2Q_{\max}C_{\text{occ}}\sqrt{\delta_t/2}.$$

□

A.2 Lemma 2: Two-phase regret decomposition

Lemma 2 (Two-phase regret decomposition)**.** *Let $T = T_1 + T_2$, and define the policy-sequence regret as*

$$\text{Reg}_T := \sum_{t=1}^{T} \left(J(\pi^*) - J(\pi_t)\right),$$

where π^* is an optimal policy. In phase I ($1 \le t \le T_1$), suppose ANCHOR uses KL anchoring satisfying

$$D_{d^{\pi_{\text{off}}}}(\pi_t \| \pi_{\text{off}}) \le \delta_t.$$

In phase II ($T_1 + 1 \le t \le T$), suppose the base online learner satisfies

$$\sum_{t=T_1+1}^{T} (J(\pi^*) - J(\pi_t)) \le \mathrm{Reg}_{\mathrm{base}}(T_2).$$

Assume that

$$|A^{\pi_{\mathrm{off}}}(s,a)| \le A_{\max} \qquad \textit{for all } (s,a),$$

and define

$$\Delta_{\mathrm{off}} := J(\pi^*) - J(\pi_{\mathrm{off}}).$$

Then under Assumption 1,

$$\mathrm{Reg}_T \le T_1 \Delta_{\mathrm{off}} + \frac{A_{\max} C_{\mathrm{occ}}}{1-\gamma} \sum_{t=1}^{T_1} \sqrt{\delta_t/2} + \mathrm{Reg}_{\mathrm{base}}(T_2). \tag{9}$$

Proof. By definition,

$$\mathrm{Reg}_T = \sum_{t=1}^{T} \big(J(\pi^*) - J(\pi_t)\big) = \sum_{t=1}^{T_1} \big(J(\pi^*) - J(\pi_t)\big) + \sum_{t=T_1+1}^{T} \big(J(\pi^*) - J(\pi_t)\big).$$

For phase I, add and subtract $J(\pi_{\mathrm{off}})$:

$$J(\pi^*) - J(\pi_t) = \Delta_{\mathrm{off}} + \big(J(\pi_{\mathrm{off}}) - J(\pi_t)\big).$$

Thus it remains to bound $|J(\pi_t) - J(\pi_{\mathrm{off}})|$.

By the performance difference lemma (5),

$$J(\pi_t) - J(\pi_{\mathrm{off}}) = \frac{1}{1-\gamma} \mathbb{E}_{(s,a)\sim d^{\pi_t}} \left[A^{\pi_{\mathrm{off}}}(s,a)\right].$$

Also,

$$\mathbb{E}_{(s,a)\sim d^{\pi_{\mathrm{off}}}} [A^{\pi_{\mathrm{off}}}(s,a)] = 0,$$

because

$$\sum_a \pi_{\mathrm{off}}(a|s) A^{\pi_{\mathrm{off}}}(s,a) = 0 \qquad \text{for every state } s.$$

Therefore,

$$J(\pi_t) - J(\pi_{\mathrm{off}}) = \frac{1}{1-\gamma} \sum_{s,a} \big(d^{\pi_t}(s,a) - d^{\pi_{\mathrm{off}}}(s,a)\big) A^{\pi_{\mathrm{off}}}(s,a).$$

Using $|A^{\pi_{\mathrm{off}}}(s,a)| \le A_{\max}$,

$$|J(\pi_t) - J(\pi_{\mathrm{off}})| \le \frac{A_{\max}}{1-\gamma} \|d^{\pi_t} - d^{\pi_{\mathrm{off}}}\|_1.$$

By Assumption 1 and Pinsker,

$$\|d^{\pi_t} - d^{\pi_{\mathrm{off}}}\|_1 \le C_{\mathrm{occ}} \, \mathrm{TV}_{d^{\pi_{\mathrm{off}}}}(\pi_t, \pi_{\mathrm{off}}) \le C_{\mathrm{occ}} \sqrt{\delta_t/2}.$$

Hence

$$J(\pi^*) - J(\pi_t) \le \Delta_{\mathrm{off}} + \frac{A_{\max} C_{\mathrm{occ}}}{1-\gamma} \sqrt{\delta_t/2}.$$

Summing over $t = 1, \ldots, T_1$ yields

$$\sum_{t=1}^{T_1} \big(J(\pi^*) - J(\pi_t)\big) \le T_1 \Delta_{\mathrm{off}} + \frac{A_{\max} C_{\mathrm{occ}}}{1-\gamma} \sum_{t=1}^{T_1} \sqrt{\delta_t/2}.$$

For phase II, apply the assumed regret guarantee of the base learner:

$$\sum_{t=T_1+1}^{T} \big(J(\pi^*) - J(\pi_t)\big) \le \mathrm{Reg}_{\mathrm{base}}(T_2).$$

Combining the two parts proves (9). □

B Other Attempts to Mitigate Catastrophic Failure

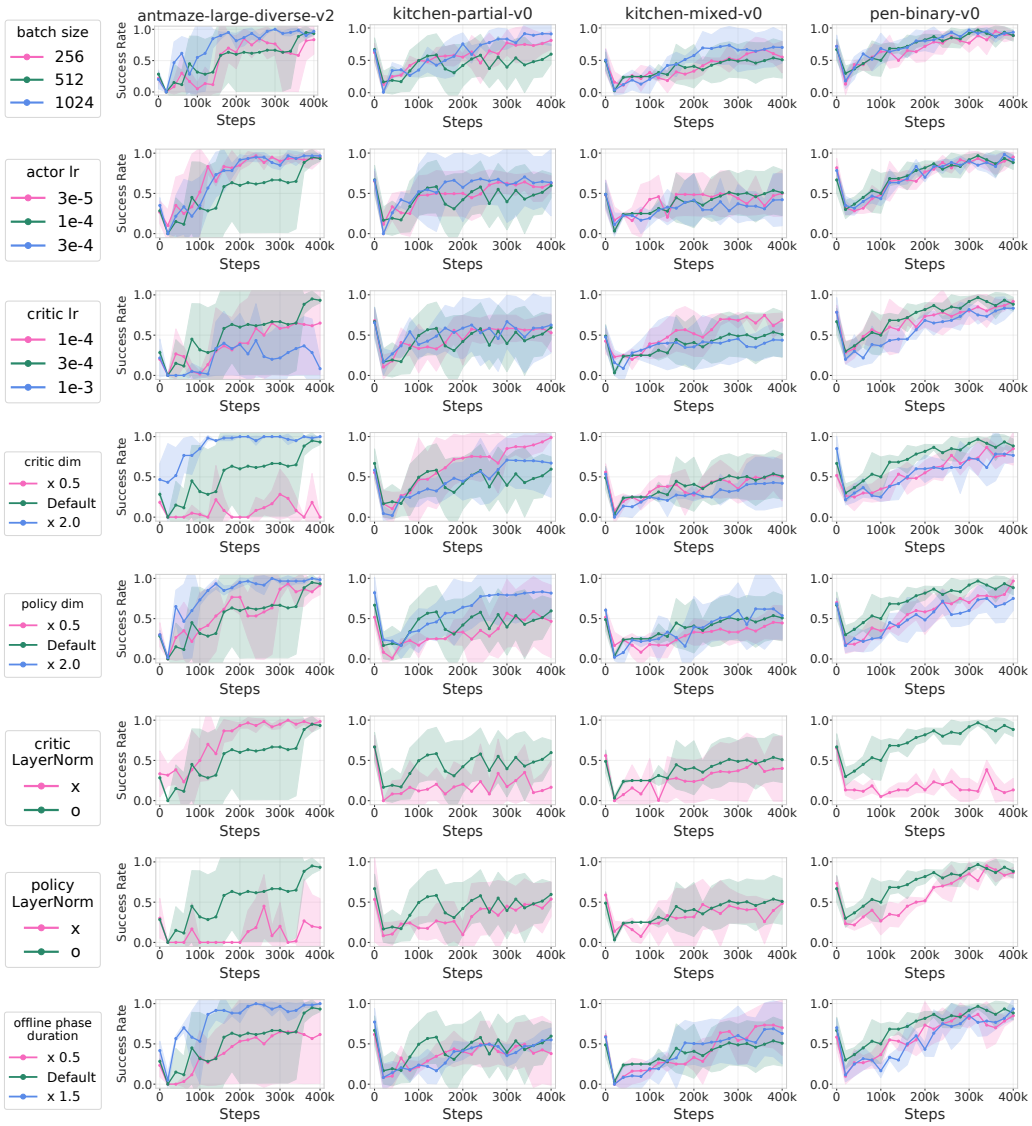


Figure 8: **Hyperparameter and architectural tweaks do not consistently mitigate catastrophic failure.** From top to bottom, the plots correspond to: batch size, actor learning rate, critic learning rate, critic hidden dimension, actor hidden dimension, LayerNorm in the critic, LayerNorm in the actor, and the duration of the offline pretraining phase. Across all settings and tasks, none of these factors consistently mitigates catastrophic failure.

In Section 3.2, we examined whether tuning the UTD ratio, varying warmup length, adjusting the Cal-QL conservative weight $\alpha_{\text{Cal-QL}}$, or regularizing the critic shift via $D_{\text{KL}}\left(\text{softmax}(Q^{\text{offline}}) \,\|\, \text{softmax}(Q^{\text{online}})\right)$ could mitigate catastrophic failure. None of these interventions proved effective. Here, we extend this investigation to additional settings: batch size, actor learning rate, critic learning rate, critic layer hidden dimension size, actor layer hidden dimension size, LayerNorm in the critic, LayerNorm in the actor, and the duration of the offline pretraining phase. As shown in Figure 8, none of these choices consistently reduces catastrophic failure across the four tasks, reinforcing our conclusion that such collapses are not reliably resolved by existing stabilization strategies.

C Can Q-Driven Metrics Explain Catastrophic Failure?

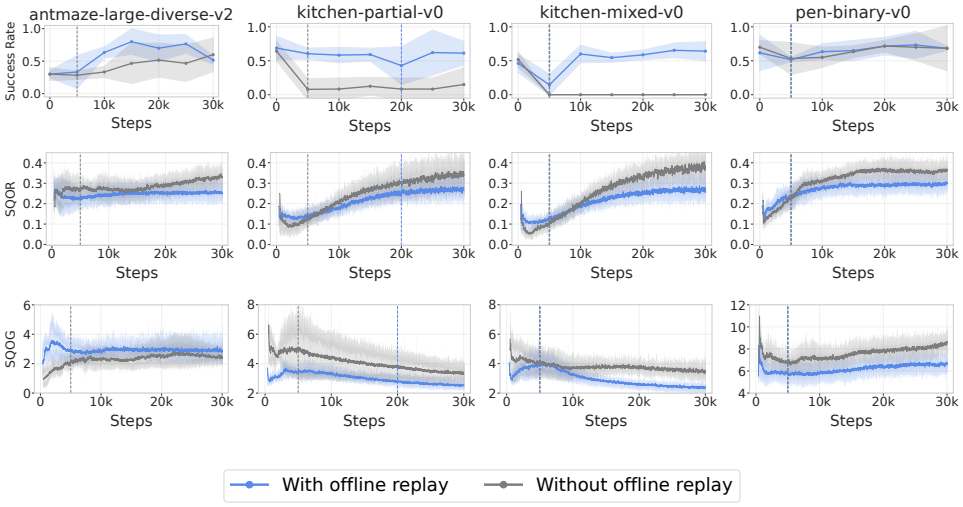


Figure 9: **SQOR and SQOG do not reliably predict catastrophic failure.** We compare success rate, the spurious Q-optimism ratio (SQOR), and the spurious Q-optimism gap (SQOG) across four tasks. Both metrics show task-dependent and inconsistent alignment with early performance collapse. The vertical dashed line marks the timestep at which the largest performance drop occurs.

In Section 3.2, we showed that explicitly slowing the critic's distributional shift does not reliably prevent catastrophic failure. Here, we further examine whether other Q-driven diagnostics can explain or predict early collapse. We focus on the setting in Figure 1(b), where online fine-tuning proceeds without offline replay while retaining the conservative regularizer.

We evaluate four tasks: antmaze-large-diverse, kitchen-partial, kitchen-mixed, and pen-binary. Since catastrophic failure typically occurs early in online fine-tuning, we report all metrics over the first 30K online steps. Due to computational constraints, this analysis is run with three random seeds.

For a minibatch of states s, let $a_t \sim \pi_t(\cdot|s)$ denote an action sampled from the current online policy and let $a_{\text{off}} \sim \pi_{\text{off}}(\cdot|s)$ denote an action sampled from the offline-pretrained policy. Let Q_t denote the critic at online step t, and let Q_{final} denote the critic checkpoint after 400K online steps for the same random seed. We define

$$\Delta_t(s) = Q_t(s, a_t) - Q_t(s, a_{\text{off}}), \qquad \Delta_{\text{final}}(s) = Q_{\text{final}}(s, a_t) - Q_{\text{final}}(s, a_{\text{off}}).$$

Intuitively, Δ_t measures the current critic's relative preference between the online-policy action and the offline-policy action, while Δ_{final} provides a post-hoc reference after online fine-tuning has stabilized.

Spurious Q-optimism ratio and mismatch magnitude. We first examine two preference-based Q diagnostics. Let $\mathcal{B}_t$ denote the minibatch of states sampled from the online replay buffer at step t, and let $\text{sgn}(\cdot)$ denote the sign function. The first metric is the spurious Q-optimism ratio:

$$\text{SQOR}(t) = \frac{1}{|\mathcal{B}_t|} \sum_{s \in \mathcal{B}_t} \mathbf{1}\left[\Delta_t(s) > 0 \ \wedge \ \Delta_{\text{final}}(s) < 0\right].$$

SQOR measures the fraction of batch states where the current critic favors the current-policy action, while the final critic favors the offline-policy action. The second metric measures the magnitude of preference mismatch. Define the set of mismatched states within the current online batch as

$$\mathcal{M}_t = \{s \in \mathcal{B}_t : \text{sgn}(\Delta_t(s)) \neq \text{sgn}(\Delta_{\text{final}}(s))\}.$$

We define the spurious Q-optimism gap as

$$\text{SQOG}(t) = \frac{1}{|\mathcal{M}_t|} \sum_{s \in \mathcal{M}_t} \left|\Delta_t(s) - \Delta_{\text{final}}(s)\right|,$$

with the value set to zero when $\mathcal{M}_t$ is empty.

As shown in Figure 9, neither SQOR nor SQOG consistently tracks catastrophic failure across tasks and stress settings. In some cases, the metrics increase without a corresponding collapse; in others, early performance drops are not preceded by a clear rise in either metric. This suggests that these preference-based Q diagnostics alone do not provide a reliable explanation for catastrophic failure.

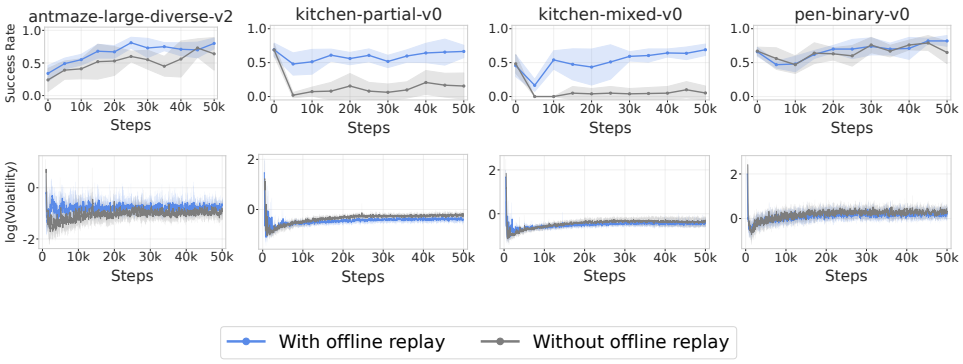


Figure 10: **Q-update volatility does not reliably predict catastrophic failure.** We compare success rate and critic-update volatility across four tasks. Volatility shows weak and inconsistent correspondence with early performance collapse.

Q-update volatility. We also examine whether abrupt critic updates explain early collapse. We define volatility as the square root of the bias-corrected exponential moving average of squared one-step Q-updates [Kingma and Ba, 2014]:

$$\text{Volatility}_t = \sqrt{\frac{m_t}{1-\beta^t}}, \qquad m_t = \beta m_{t-1} + (1-\beta)\mathbb{E}_{(s,a)\sim\mathcal{B}_t}\left[(\Delta Q_t(s,a))^2\right],$$

where $\Delta Q_t(s,a) = Q_{t+1}(s,a) - Q_t(s,a)$ and $\beta = 0.9$. This metric measures the aggregate magnitude of critic changes over minibatch state-action pairs.

Figure 10 shows that Q-update volatility also fails to consistently align with catastrophic failure. High volatility does not always lead to collapse, and severe collapse can occur without a distinctive volatility pattern. Together with the critic-shift regularization results in Section 3.2, these analyses do not rule out critic instability as a contributing factor. Rather, they suggest that critic-side stabilization alone may be insufficient to explain or prevent catastrophic failure, motivating our focus on policy-side stabilization through KL anchoring.

D Additional Ablation Studies

D.1 Sensitivity to Batch Size and UTD Ratio

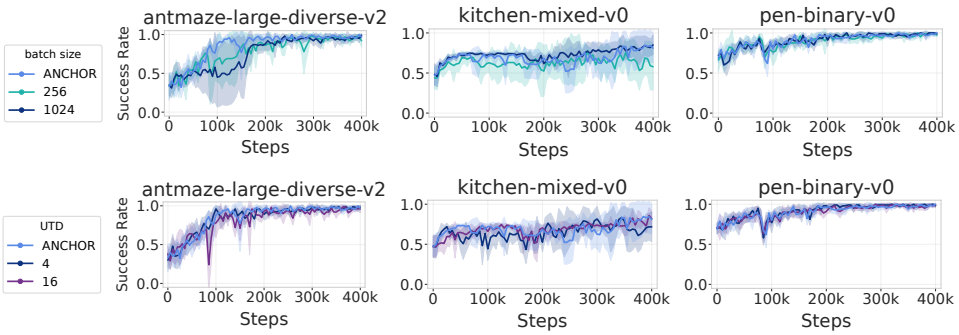


Figure 11: **Sensitivity to batch size and UTD ratio.** We vary the batch size (top) and UTD ratio (bottom) across three tasks. ANCHOR is generally robust to these choices, although larger batch size and larger UTD can increase learning instability.

Figure 11 further evaluates the sensitivity of ANCHOR to batch size and UTD ratio. Overall, ANCHOR remains robust across the tested variants in kitchen-mixed and pen-binary, with similar early stability and asymptotic performance. In antmaze-large-diverse, however, the larger batch size of 1024 and the larger UTD ratio of 16 lead to greater learning instability and weaker prevention of catastrophic failure. Nevertheless, compared with the baselines in Figure 4, these variants still achieve competitive catastrophic failure and asymptotic performance. These results indicate that ANCHOR is not highly sensitive to batch size or UTD ratio, although overly aggressive update configurations can reduce early stability.

D.2 A Larger Initial KL Weight Yields Stronger Early Anchoring

We check whether increasing the KL coefficient actually reduces the measured deviation between the online actor and the offline-pretrained policy. This is not automatic in practice, since the KL term is optimized jointly with the online RL objective and may not be minimized effectively under online updates. We compare two initial KL weights, $\beta_0 = 0.5$ and $\beta_0 = 2.0$. Figure 12 reports the KL loss during online fine-tuning.

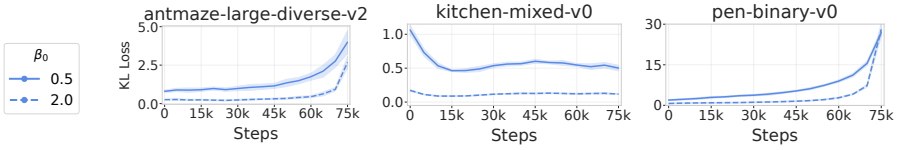


Figure 12: **Effect of the initial KL weight.** A larger initial KL weight ($\beta_0 = 2.0$) produces lower KL loss early in online fine-tuning, indicating that the KL penalty more strongly constrains deviation from the offline policy. As the KL coefficient is annealed, the gap between $\beta_0 = 0.5$ and $\beta_0 = 2.0$ gradually narrows.

Across all three tasks, the larger initial KL weight consistently produces lower KL loss during the early phase of online adaptation. This verifies that β_0 controls the effective strength of actor anchoring in practice: increasing β_0 keeps the online actor closer to the offline-pretrained policy despite simultaneous optimization of the RL objective. As training proceeds and the KL coefficient is annealed, the KL losses under the two settings become increasingly similar. This behavior is consistent with the design of ANCHOR, where KL anchoring is strongest early in online fine-tuning and gradually weakens as standard online improvement takes over.

E Can ANCHOR Extend to Other Offline RL Backbones?

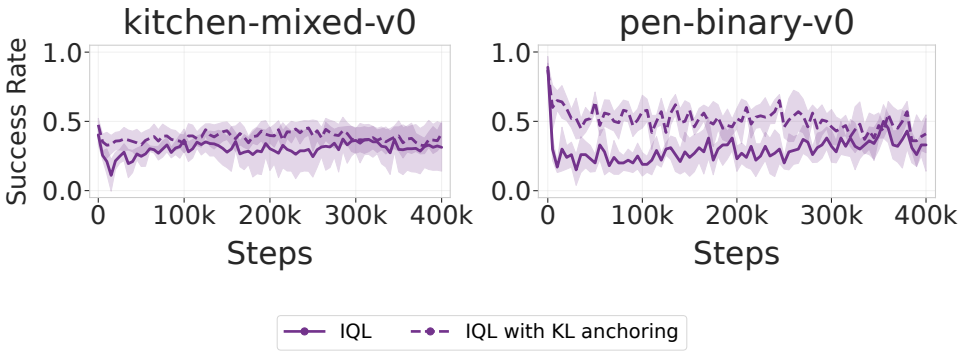


Figure 13: **Applying KL anchoring to IQL.** Adding KL anchoring with annealing to IQL reduces the initial performance drop, but does not improve asymptotic performance over the IQL baseline.

We additionally test whether the actor-side KL anchoring used in ANCHOR can be applied to another offline RL backbone. Specifically, we apply KL anchoring with annealing to IQL. Since IQL does not use an explicit CQL-style conservative regularizer, we only add the actor-side KL term toward the offline-pretrained policy and anneal its coefficient during online fine-tuning. We conduct experiments on kitchen-mixed and pen-binary, where IQL exhibits nontrivial pretrained behavior with an initial success rate above 0.4. For both tasks, we use $\beta_0 = 5.0$ and $T_{\mathrm{KL}} = 320\mathrm{K}$.

Figure 13 shows that KL anchoring can also stabilize IQL during the early online phase. Compared to naive online fine-tuning of IQL, the KL-anchored variant reduces the initial performance drop, indicating that actor-side anchoring is not specific to the CQL backbone. However, the asymptotic performance does not improve substantially over the IQL baseline. We hypothesize that this is because IQL retains its offline inductive bias during online fine-tuning, which can limit its ability to adapt to the online data distribution even when the actor is stabilized by KL anchoring. Developing backbone-specific mechanisms that remove or relax such offline biases while preserving early stability is an interesting direction for future work.

F Additional Environments and Failure Cases

We further evaluate ANCHOR in three additional environments that highlight challenging regimes for offline-to-online adaptation: kitchen-complete, relocate-binary, and antmaze-ultra-diverse. The first two environments have near-zero initial success after offline pretraining, leaving little useful pretrained behavior to preserve. In contrast, antmaze-ultra-diverse is a larger and more complex maze than antmaze-large-diverse, where sparse rewards and severe exploration mismatch make online improvement difficult. The offline pretraining duration follows the domain-specific setup used in the main experiments: 250K steps for kitchen-complete, 40K steps for relocate-binary, and 1M steps for antmaze-ultra-diverse.

For all three environments, we set the residual conservative regularization coefficient α_{on} to zero during online fine-tuning. Since kitchen-complete and relocate-binary start with near-zero success rates, we set the initial KL weight to $\beta_0 = 0$, reflecting the fact that there is little useful pretrained behavior to preserve. In kitchen-complete, ANCHOR attains the highest mean final performance and matches WSRL and PORL when confidence intervals are considered, while outperforming the remaining baselines. This behavior is consistent with Lemma 2: when the pretrained policy is weak, keeping the actor close to the offline policy can be costly, and the KL constraint should be relaxed or removed to enable online improvement. In relocate-binary, however, all methods remain near zero success throughout online fine-tuning, suggesting that this environment requires additional exploration or adaptation mechanisms beyond the scope of ANCHOR.

For antmaze-ultra-diverse, we use $\beta_0 = 1.0$ and set the KL annealing interval to $T_{\mathrm{KL}} = 400\mathrm{K}$, covering the full online fine-tuning phase. Despite this longer anchoring schedule, ANCHOR's final

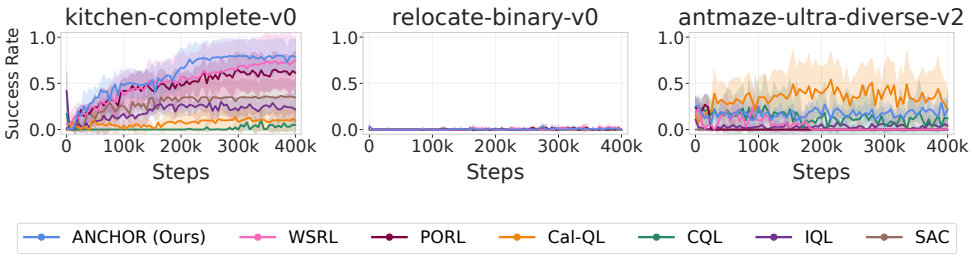


Figure 14: **Additional environments reveal limitations of KL-anchored adaptation.** In kitchen-complete, ANCHOR remains competitive despite near-zero initial success. In relocate-binary, all methods fail to make meaningful progress. In antmaze-ultra-diverse, most methods do not improve over the offline-pretrained policy, suggesting that harder sparse-reward exploration requires mechanisms beyond KL anchoring alone.

asymptotic performance remains below its initial offline-pretrained performance. Except for Cal-QL, none of the methods achieve meaningful improvement over the offline-pretrained success rate. This suggests that in extremely challenging sparse-reward environments, preserving the pretrained policy alone may be insufficient; the agent may also require exploration or online adaptation mechanisms.

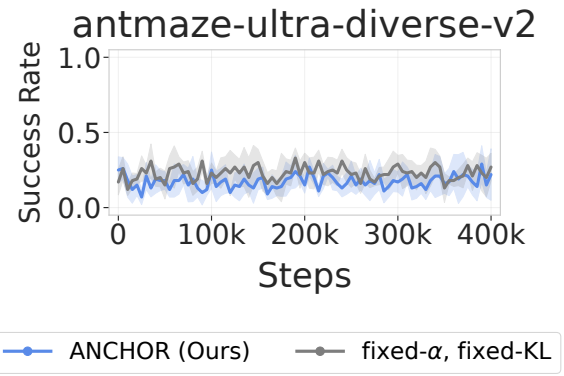


Figure 15: **Stronger preservation is insufficient in antmaze-ultra-diverse.** Keeping both the conservative coefficient and KL coefficient fixed yields a higher mean success rate than ANCHOR during online fine-tuning, but does not significantly improve asymptotic performance.

To further test whether stronger preservation of offline-pretrained behavior helps in antmaze-ultra-diverse, we evaluate an additional variant that keeps the conservative coefficient at its offline-pretraining value, $\alpha = 1.0$, and keeps the KL coefficient fixed at $\beta_t = 1.0$ without annealing. As shown in Figure 15, this variant achieves a higher mean success rate than ANCHOR across much of online fine-tuning, but its asymptotic performance does not improve significantly. These results indicate that, in extremely challenging exploration settings, simply retaining conservative regularization or maintaining a fixed KL anchor is not sufficient. Developing mechanisms for online exploration and adaptation in such environments remains an important direction for future work.

G Related Works

Offline RL. Online reinforcement learning requires interaction with the environment, which can be expensive, unsafe, or impractical in domains such as robotics, healthcare, and recommender systems [Singh et al., 2022, Liu et al., 2020, Chen et al., 2024]. Offline RL addresses this limitation by learning policies from a static dataset collected by a behavior policy [Prudencio et al., 2023]. However, because the dataset provides limited coverage of the state-action space, offline-trained policies can suffer when they encounter unseen or poorly covered regions at deployment [Prudencio et al., 2023].

A large body of work has been developed to address this challenge, including regularization-based methods [Fujimoto and Gu, 2021, Park et al., 2025, Tarasov et al., 2023, Kumar et al., 2020], uncertainty-aware methods [An et al., 2021, Nikulin et al., 2023], model-based methods [Kidambi et al., 2020, Yu et al., 2020, 2021, Chitnis et al., 2024, Park and Lee, 2025], one-step methods [Brandfonbrener et al., 2021, Eysenbach et al., 2022, Park et al., 2025], weighted-regression methods [Peng et al., 2019, Wang et al., 2020], and in-sample maximization methods [Garg et al., 2023, Kostrikov et al., 2022, Xu et al., 2023]. Among these approaches, we build on Conservative Q-Learning (CQL) [Kumar et al., 2020], which directly mitigates value overestimation by penalizing high Q-values for actions outside the dataset support. We also include Implicit Q-Learning (IQL) [Kostrikov et al., 2022] as a baseline, as it improves over the behavior policy through value-based policy extraction without explicitly evaluating out-of-distribution actions.

Offline-to-Online RL. Offline-to-online RL studies how to further improve an offline-pretrained agent through online interaction. Although offline RL methods such as CQL [Kumar et al., 2020], IQL [Kostrikov et al., 2022], and related approaches [Kostrikov et al., 2021, Tarasov et al., 2023] can be fine-tuned online, simply continuing the offline objective often yields limited improvement [Nakamoto et al., 2023]. This has motivated methods that explicitly adapt the offline-trained agent to the online learning regime.

Existing approaches include relaxing excessive conservatism in value estimates [Nakamoto et al., 2023, Luo et al., 2024, Hu et al., 2024], introducing an intermediate adaptation phase between offline pretraining and online fine-tuning [Zhou et al., 2025, Shin et al., 2025, Xiao et al., 2025], using multiple Q-functions [Lee et al., 2022, Zhao et al., 2023], tuning the update-to-data (UTD) ratio [Feng et al., 2024, Xiao et al., 2025], and incorporating uncertainty estimation [Guo et al., 2023, Wen et al., 2024]. Another line of work focuses on leveraging pretrained policies directly, rather than updating both the actor and critic [Uchendu et al., 2023, Xiao et al., 2025, Hu et al., 2023].

A key challenge in this transition is catastrophic failure: performance can drop sharply at the start of online fine-tuning due to distribution shift and unstable value learning [Wen et al., 2024, Nakamoto et al., 2023]. Prior work has addressed this issue through uncertainty estimation [Wen et al., 2024], calibration penalties [Nakamoto et al., 2023], and actor-critic alignment [Yu and Zhang, 2023]. In contrast, ANCHOR stabilizes the early online phase by anchoring the actor to the offline-pretrained policy with a KL penalty. The KL coefficient is linearly annealed to zero, allowing the method to move from conservative checkpoint adaptation to unconstrained online improvement. For the critic, ANCHOR immediately reduces the CQL coefficient to a small residual value α_{on} and keeps it fixed during online fine-tuning, avoiding excessive conservatism while retaining mild regularization against overestimation. Thus, ANCHOR addresses catastrophic failure with a simple adaptation schedule while preserving strong asymptotic performance, without requiring additional uncertainty models or complex auxiliary modules.

Trust-region and KL-regularized policy optimization. Our actor-side KL regularization is related in form to trust-region methods such as TRPO [Schulman et al., 2015], which constrain the KL divergence between consecutive policies to stabilize online policy updates. However, ANCHOR differs in both purpose and reference distribution. TRPO imposes a local trust region around the previous policy at each update to support stable on-policy improvement. In contrast, ANCHOR regularizes the online actor toward a fixed offline-pretrained policy during the early phase of offline-to-online adaptation. This anchor is not intended to enforce a monotonic policy improvement guarantee; rather, it preserves useful pretrained behavior when offline replay is unavailable and prevents abrupt policy drift into regions where the critic may be unreliable. Moreover, ANCHOR anneals the KL coefficient to zero, explicitly removing the constraint once the online actor and critic have adapted. Thus, while both approaches use KL-based control of policy updates, ANCHOR uses KL regularization as a temporary checkpoint-adaptation mechanism for offline-to-online RL, rather than as a per-update trust region for standard online RL.

H Details on Baseline Algorithms

Soft Actor-Critic (SAC) [Haarnoja et al., 2018]. SAC extends standard actor-critic methods by incorporating entropy maximization into the RL objective. Instead of maximizing only the expected return, SAC also maximizes the entropy of the policy to encourage exploration and prevent premature convergence to deterministic policies. The resulting objective is to maximize the expected sum of

rewards and entropies:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \left(r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right) \right],$$

where $\mathcal{H}(\pi(\cdot|s_t)) = -\mathbb{E}_{a_t \sim \pi}[\log \pi(a_t|s_t)]$ denotes the entropy of the policy at state s_t, and $\alpha > 0$ is the temperature parameter that balances reward maximization and entropy.

Conservative Q-Learning (CQL) [Kumar et al., 2020]. CQL learns a Q-function by explicitly regularizing Q-values to mitigate overestimation issues common in offline RL [Prudencio et al., 2023]:

$$\mathcal{L}(\theta) = \alpha \underbrace{\left(\mathbb{E}_{s \sim \mathcal{D}, a \sim \pi}[Q_\theta(s, a)] - \mathbb{E}_{s,a \sim \mathcal{D}}[Q_\theta(s, a)] \right)}_{\text{Conservative regularizer}} + \mathcal{L}_{\text{TD}}(\theta)$$

Here, π represents the current policy, $\mathcal{D}$ is the offline dataset, and α controls the intensity of the conservative regularization. The term $\mathcal{L}_{TD}$ is the temporal difference loss used in Q-learning methods, while the conservative regularization penalizes Q-values for state-action pairs not present in the offline dataset $\mathcal{D}$.

Implicit Q-learning (IQL) [Kostrikov et al., 2022]. The IQL algorithm learns a state-value function $V_{\theta_V} : \mathcal{S} \to \mathbb{R}$ and an action-value function $Q_{\theta_Q} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ by minimizing:

$$\mathcal{L}_V(\theta_V) = \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\ell_\kappa^2 \left(V_{\theta_V}(s) - Q_{\bar{\theta}_Q}(s, a) \right) \right],$$

$$\mathcal{L}_Q(\theta_Q) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_{\theta_Q}(s, a) - r - \gamma V_{\theta_V}(s') \right)^2 \right],$$

where the expectile loss is defined as $\ell_\kappa^2(x) = |\kappa - \mathbf{1}[x < 0]|x^2$ [Newey and Powell, 1987], and $\bar{\theta}_Q$ represents the target network parameters. Unlike standard MSE loss, expectile loss asymmetrically weights positive and negative errors, with $\kappa > 0.5$ emphasizing higher Q-values. By doing so, IQL evaluates only actions from the dataset, which enables policy improvement without querying out-of-distribution actions, thereby inducing implicit conservatism.

Calibrated Q-Learning (Cal-QL) [Nakamoto et al., 2023]. Cal-QL adjusts CQL by calibrating the learned Q-function relative to a reference policy to avoid initial degradation caused by overly pessimistic Q-values during fine-tuning. The conservative regularization term in CQL is altered by substituting $\mathbb{E}_{s \sim \mathcal{D}, a \sim \pi}[Q_\theta(s, a)]$ with $\mathbb{E}_{s \sim \mathcal{D}, a \sim \pi}[\max(Q_\theta(s, a), V^\mu(s))]$, where $V^\mu(s)$ represents the value function of the reference policy μ, which can be estimated via Monte-Carlo return. This calibration prevents the conservative penalty from further suppressing actions whose estimated values are already below the reference value, thereby mitigating the overly pessimistic Q-values that can hinder online fine-tuning.

Policy-Only Reinforcement Learning Fine-Tuning (PORL) [Xiao et al., 2025]. PORL focuses on the challenge of fine-tuning online RL using only a pretrained policy, without relying on pretrained Q-functions or offline datasets. This approach is especially beneficial when pretrained Q-functions are either unreliable due to pessimism or unavailable, such as in imitation learning scenarios. PORL begins by training the randomly initialized Q-function at the beginning of the online fine-tuning phase. Training data is collected based on the online interaction of the pretrained policy using an epsilon-greedy exploration strategy. During this initial sampling phase, the Q-function is trained via temporal difference learning with a high UTD ratio. After this pre-sampling period, PORL transitions to standard SAC fine-tuning, updating both the policy and the Q-function.

Warm-start RL (WSRL) [Zhou et al., 2025]. WSRL addresses the distribution shift issue from offline to online data in fine-tuning without offline data by introducing a warmup phase. This phase utilizes a frozen pretrained policy to collect online rollouts. The data collected during warmup bridges the distribution mismatch and helps recalibrate the offline Q-function to the online distribution, allowing the method to adapt in the online environment quickly. Once the warmup phase is complete, WSRL proceeds with conventional online RL using SAC, employing a high UTD ratio.

I Implementation Details

I.1 Details on Environments

AntMaze. The AntMaze environment, part of the D4RL benchmark suite [Fu et al., 2020], tasks an 8-DoF ant quadruped robot with navigating through a large and complex maze to reach a designated goal position. The agent receives a binary reward of +1 only upon successfully reaching the goal. The observation space is 29-dimensional, including the robot's position, orientation, and velocity, while the action space is a continuous 8-dimensional vector, normalized to the range $[-1, 1]$. We use three variants: antmaze-large-diverse-v2, which contains trajectories collected by commanding the agent to random goals from random start positions; antmaze-large-play-v2, which contains trajectories directed to a specific location; and antmaze-ultra-diverse-v2, which uses a larger and more complex maze with the same diverse data collection strategy. The two large-maze variants share a maximum episode length of 1000 steps, while antmaze-ultra-diverse-v2 allows up to 700 steps.

FrankaKitchen. The FrankaKitchen environment contained in D4RL [Fu et al., 2020] requires controlling a 9-DoF Franka Panda robotic arm to manipulate various kitchen appliances and configure the environment into a predefined target state. Each task consists of four subtasks, and the agent receives a reward between 0 and 4 based on the number of successfully completed subtasks. The observation space is 60-dimensional, encompassing joint positions and object states, and the action space is a 9-dimensional continuous vector normalized to $[-1, 1]$. We use three benchmark environments from D4RL: kitchen-partial-v0, kitchen-mixed-v0, and kitchen-complete-v0. kitchen-partial-v0 includes a mix of complete and incomplete demonstrations, where task elements involve operating the microwave, kettle, light switch, and slide cabinet. kitchen-mixed-v0 contains only incomplete demonstrations, with task components including the microwave, kettle, bottom burner, and light switch. kitchen-complete-v0 contains only complete demonstrations covering the same four subtasks as kitchen-partial-v0: microwave, kettle, light switch, and slide cabinet. All three environments use a maximum episode length of 280 steps.

Adroit. The Adroit suite [Rajeswaran et al., 2017] evaluates dexterous manipulation using a high-DoF robotic hand. Specifically, we use three tasks from the D4RL benchmark: pen-binary-v0, door-binary-v0, and relocate-binary-v0. In pen-binary-v0, a 24-DoF shadow hand must reorient a pen to match a target pose, while in door-binary-v0, a 28-DoF hand must grasp and rotate a door handle to open it. In relocate-binary-v0, a 30-DoF hand must pick up an object and relocate it to a target position. All three environments have sparse binary rewards: a reward of +1 is given only upon successful task completion. The observation space is 45-dimensional for the pen task, 39-dimensional for the door and relocate tasks, consisting of hand joint angles and object poses. The action space is continuous, 24-dimensional for the pen task, 28-dimensional for the door task, and 30-dimensional for the relocate task, each normalized to $[-1, 1]$. The maximum episode lengths are 100 steps for the pen task and 200 steps for the door and relocate tasks.

I.2 Network Architectures

ANCHOR uses the same network architecture as the CQL baseline in the WSRL codebase [Zhou et al., 2025]. The agent follows an actor-critic framework. The actor takes an observation as input and outputs the mean and log standard deviation of a Gaussian action distribution. The critic takes the concatenated observation-action pair as input and uses an ensemble of ten Q-functions, each producing a scalar Q-value. For stability, two Q-functions are randomly subsampled during target computation, and the minimum of the two values is used. Both the actor and critic are implemented as multilayer perceptrons (MLPs) with ReLU activations. A learnable temperature parameter controls the entropy regularization. The hidden dimensions for each domain are reported in Table 1.

I.3 Reproducing Baselines

All baseline methods are reproduced under a unified experimental setup. For WSRL, Cal-QL, CQL, IQL, and SAC, we use the implementations and hyperparameters provided in the WSRL codebase [Zhou et al., 2025], with several stability-oriented modifications applied consistently across methods. Specifically, (i) we increase the batch size from 256 to 512; (ii) we use an ensemble of ten Q-functions and compute target values using the minimum over two critics randomly subsampled at each update, following REDQ [Chen et al., 2021]; and (iii) we apply layer normalization [Ba et al.,

2016] to both the policy and critic networks to mitigate uncontrolled extrapolation effects [Ball et al., 2023]. Since no public implementation of PORL was available, we reimplement it following Xiao et al. [2025] and use the authors' reported settings.

I.4 Hyperparameters

Table 1 reports the hyperparameters used to train ANCHOR. For hyperparameters not specific to ANCHOR, we follow the default CQL settings used by Zhou et al. [2025], except for the UTD ratio in door-binary, where we use the WSRL setting. For ANCHOR-specific hyperparameters, we tune $\alpha_{\text{on}} \in \{0.0, 1.0\}$, $T_{\text{KL}} \in \{40\text{K}, 80\text{K}, 160\text{K}\}$, and $\beta_0 \in \{0.2, 0.4, 1.0\}$.

Table 1: **Hyperparameters of ANCHOR.**

	AntMaze		**Adroit**		**Kitchen**	
	large diverse	large play	door	pen	partial	mixed
ANCHOR						
α_{on}	0.0	0.0	0.0	0.0	1.0	1.0
T_{KL}	80,000	80,000	40,000	80,000	160,000	160,000
β_0	0.4	0.2	0.2	1.0	1.0	1.0
Optimization						
Actor Learning Rate	1×10^{-4}		1×10^{-4}		1×10^{-4}	
Critic Learning Rate	3×10^{-4}		3×10^{-4}		3×10^{-4}	
Batch Size	512		512		512	
UTD	1	1	16	1	1	1
Offline Steps	1,000,000		40,000		250,000	
α_{off}	5.0		1.0		5.0	
Warmup Steps	5,000		5,000		5,000	
Architecture						
Critic Network	[256, 256, 256, 256]		[512, 512, 512]		[512, 512, 512]	
Actor Network	[256, 256]		[512, 512]		[512, 512, 512]	
Activations	ReLU		ReLU		ReLU	
Q-ensemble	10		10		10	

I.5 Online Training Time Comparison

Table 2 compares the online training time of ANCHOR and the baselines. ANCHOR has a similar average training time to WSRL and PORL, despite the additional actor-side KL regularization. While it is slower than lightweight baselines such as IQL and SAC, these methods suffer from substantially weaker asymptotic performance and/or larger catastrophic failure in our main results. Considering the gains of ANCHOR in both early stability and final performance, its computational overhead is moderate and comparable to other competitive offline-to-online methods.

Table 2: **Online training time (in decimal hours) comparison across baselines.**

Task	**ANCHOR (Ours)**	WSRL	PORL	Cal-QL	CQL	IQL	SAC
antmaze-large-diverse-v2	2.75 ± 0.02	2.73 ± 0.01	3.35 ± 0.07	3.44 ± 0.01	2.82 ± 0.02	2.08 ± 0.02	1.53 ± 0.0
antmaze-large-play-v2	2.75 ± 0.01	3.48 ± 0.08	3.54 ± 0.06	3.26 ± 0.03	2.34 ± 0.03	2.09 ± 0.03	1.52 ± 0.0
kitchen-partial-v0	4.31 ± 0.02	3.37 ± 0.0	3.41 ± 0.03	4.35 ± 0.01	3.28 ± 0.05	1.41 ± 0.01	1.38 ± 0.0
kitchen-mixed-v0	3.88 ± 0.03	3.61 ± 0.01	3.51 ± 0.13	4.36 ± 0.0	3.33 ± 0.02	1.46 ± 0.02	1.36 ± 0.0
door-binary-v0	3.07 ± 0.01	2.9 ± 0.01	2.93 ± 0.01	3.55 ± 0.01	2.43 ± 0.01	1.28 ± 0.02	0.78 ± 0.01
pen-binary-v0	2.33 ± 0.02	2.77 ± 0.01	2.79 ± 0.01	3.51 ± 0.01	2.29 ± 0.01	1.13 ± 0.1	0.67 ± 0.0
Average	3.18	3.14	3.25	3.75	2.75	1.58	1.21

I.6 Experimental Setup and Reproducibility

All experiments were conducted on a compute node equipped with 8 NVIDIA A100 GPUs (80GB each), an AMD EPYC 7543 32-Core CPU, and 885 GB of RAM. The software environment was based on Python 3.10.18, Pytorch 2.7.0, JAX 0.4.25 with CUDA 12.2.

J Limitations

While ANCHOR provides a simple and effective recipe for offline-to-online adaptation without offline replay, several limitations remain.

Scope of evaluation. Our experiments focus on single-agent continuous-control benchmarks, including AntMaze, FrankaKitchen, and Adroit. We do not evaluate ANCHOR under hard safety constraints, non-stationary dynamics, partial observability, or multi-agent settings. Therefore, its effectiveness in these regimes is not guaranteed.

Dependence on pretrained policy quality. ANCHOR is most effective when the offline-pretrained policy contains useful behavior to preserve. When the pretrained policy has near-zero success, as in relocate-binary, KL anchoring provides limited benefit. This suggests that ANCHOR is better suited for stabilizing useful pretrained behavior than for solving tasks where the offline policy provides little meaningful guidance.

Limited exploration in hard sparse-reward environments. ANCHOR controls policy drift during online fine-tuning, but it does not introduce a new exploration mechanism. In challenging sparse-reward environments such as antmaze-ultra-diverse, most methods fail to improve substantially over the offline-pretrained policy. Addressing such settings may require combining KL anchoring with stronger exploration or task-specific adaptation mechanisms.

Hand-designed KL schedules. ANCHOR uses a manually specified initial KL weight and annealing interval. Although our ablations show robustness to moderate changes in these hyperparameters, selecting the schedule still requires some environment-level tuning. Future work could develop adaptive KL schedules based on online uncertainty, critic reliability, or measured policy drift.

Backbone dependence. Most of our experiments instantiate ANCHOR on top of CQL. We also test actor-side KL anchoring with IQL and find that it reduces early catastrophic failure, but does not improve asymptotic performance. This suggests that different offline RL backbones may require backbone-specific mechanisms for relaxing offline inductive biases during online adaptation.

Statistical coverage. Unless otherwise noted, experiments are run with five random seeds due to computational constraints, and some auxiliary analyses use fewer seeds. Although confidence intervals are reported, rare failure modes may still be under-sampled.

K Visualizations



(a) **antmaze-large-diverse-v2:** 8-DoF ant navigating through a maze to reach the goal.



(b) **antmaze-large-play-v2:** Same evaluation as diverse-v2, different training data.



(c) **antmaze-ultra-diverse-v2:** 8-DoF ant navigating through a larger maze than large-diverse-v2.



(d) **kitchen-partial-v0:** Completing subtasks: microwave, kettle, light switch, and slide cabinet.



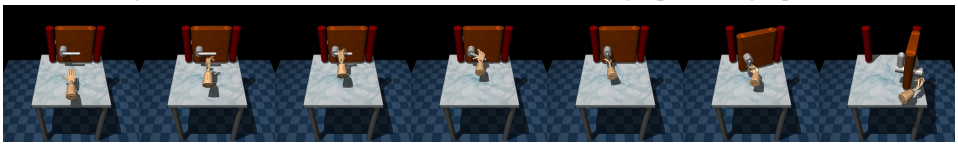
(e) **kitchen-mixed-v0:** Completing subtasks: microwave, kettle, bottom burner, and light switch.



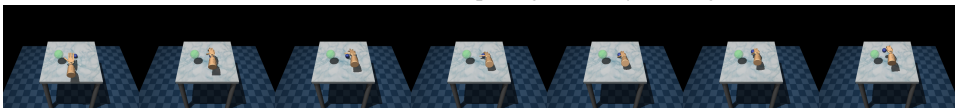
(f) **kitchen-complete-v0:** Completing subtasks: microwave, kettle, light switch, and slide cabinet.



(g) **pen-binary-v0:** 24-DoF shadow hand reorienting a pen to target pose.



(h) **door-binary-v0:** 28-DoF hand opening a door by rotating the handle.



(i) **relocate-binary-v0:** 30-DoF shadow hand relocating an object to a target location.

Figure 16: **Representative episodes across nine tasks.** Each row shows temporally ordered frames from a trajectory, demonstrating navigation (antmaze-ultra-diverse-v2, antmaze-large-diverse-v2, antmaze-large-play-v2), kitchen manipulation (kitchen-partial-v0, kitchen-mixed-v0, kitchen-complete-v0), and dexterous control (pen-binary-v0, door-binary-v0, relocate-binary-v0).

L Numerical Results

L.1 Raw Performance and Catastrophic Failure Values

Table 3: **Performance at 400k online steps. Bold** indicates the best performance per task, and underline indicates the second best.

Task	**ANCHOR (Ours)**	WSRL	PORL	Cal-QL	CQL	IQL	SAC
antmaze-large-diverse-v2	**0.97** $\pm$ 0.01	0.68 $\pm$ 0.17	0.9 $\pm$ 0.07	**0.97** $\pm$ 0.01	0.92 $\pm$ 0.05	0.06 $\pm$ 0.05	0.0 $\pm$ 0.0
antmaze-large-play-v2	**0.96** $\pm$ 0.02	0.95 $\pm$ 0.02	0.47 $\pm$ 0.2	0.9 $\pm$ 0.07	0.95 $\pm$ 0.03	0.01 $\pm$ 0.01	0.0 $\pm$ 0.0
kitchen-partial-v0	**1.0** $\pm$ 0.0	0.93 $\pm$ 0.06	0.91 $\pm$ 0.06	0.7 $\pm$ 0.09	0.44 $\pm$ 0.12	0.4 $\pm$ 0.05	0.49 $\pm$ 0.07
kitchen-mixed-v0	0.81 $\pm$ 0.09	0.72 $\pm$ 0.11	**0.88** $\pm$ 0.06	0.45 $\pm$ 0.12	0.4 $\pm$ 0.06	0.31 $\pm$ 0.09	0.3 $\pm$ 0.05
door-binary-v0	**1.0** $\pm$ 0.0	0.8 $\pm$ 0.2	0.6 $\pm$ 0.24	0.19 $\pm$ 0.19	0.18 $\pm$ 0.18	0.0 $\pm$ 0.0	0.2 $\pm$ 0.2
pen-binary-v0	**1.0** $\pm$ 0.0	0.97 $\pm$ 0.01	0.99 $\pm$ 0.01	0.94 $\pm$ 0.02	0.81 $\pm$ 0.09	0.33 $\pm$ 0.1	0.78 $\pm$ 0.04
Average	**0.86** $\pm$ 0.04	0.72 $\pm$ 0.07	0.68 $\pm$ 0.07	0.62 $\pm$ 0.06	0.55 $\pm$ 0.07	0.16 $\pm$ 0.03	0.25 $\pm$ 0.06

Table 4: **Performance at 350k online steps. Bold** indicates the best performance per task, and underline indicates the second best.

Task	**ANCHOR (Ours)**	WSRL	PORL	Cal-QL	CQL	IQL	SAC
antmaze-large-diverse-v2	**0.97** $\pm$ 0.01	0.73 $\pm$ 0.18	0.84 $\pm$ 0.07	0.95 $\pm$ 0.03	0.96 $\pm$ 0.02	0.03 $\pm$ 0.03	0.0 $\pm$ 0.0
antmaze-large-play-v2	0.96 $\pm$ 0.02	**0.98** $\pm$ 0.01	0.37 $\pm$ 0.23	0.89 $\pm$ 0.07	0.95 $\pm$ 0.03	0.01 $\pm$ 0.01	0.0 $\pm$ 0.0
kitchen-partial-v0	**1.0** $\pm$ 0.0	0.95 $\pm$ 0.05	0.9 $\pm$ 0.06	0.7 $\pm$ 0.09	0.44 $\pm$ 0.12	0.39 $\pm$ 0.05	0.47 $\pm$ 0.06
kitchen-mixed-v0	0.83 $\pm$ 0.1	0.7 $\pm$ 0.12	**0.88** $\pm$ 0.06	0.45 $\pm$ 0.12	0.4 $\pm$ 0.06	0.3 $\pm$ 0.08	0.25 $\pm$ 0.03
door-binary-v0	**0.97** $\pm$ 0.01	0.8 $\pm$ 0.2	0.6 $\pm$ 0.24	0.17 $\pm$ 0.17	0.19 $\pm$ 0.19	0.0 $\pm$ 0.0	0.2 $\pm$ 0.2
pen-binary-v0	0.94 $\pm$ 0.02	**0.96** $\pm$ 0.02	0.95 $\pm$ 0.02	**0.96** $\pm$ 0.01	0.72 $\pm$ 0.05	0.4 $\pm$ 0.05	0.84 $\pm$ 0.02
Average	**0.83** $\pm$ 0.05	0.72 $\pm$ 0.07	0.65 $\pm$ 0.07	0.63 $\pm$ 0.06	0.54 $\pm$ 0.06	0.16 $\pm$ 0.03	0.25 $\pm$ 0.06

Tables 3 and 4 report success rates (mean $\pm$ SE) after 400k and at 350k online fine-tuning, respectively, averaged over five seeds. Success rates for all baselines stabilize by 350k steps, validating the 350k-400k window as a reliable proxy for asymptotic performance. Offline-to-online methods consistently outperform offline-only approaches, and ANCHOR achieves the highest average success rate.

Table 5: **Performance after the offline phase.**

Task	**ANCHOR (Ours)**	WSRL	PORL	Cal-QL	CQL	IQL
antmaze-large-diverse-v2	0.33 $\pm$ 0.09	0.24 $\pm$ 0.05	0.39 $\pm$ 0.04	0.26 $\pm$ 0.04	0.26 $\pm$ 0.05	0.03 $\pm$ 0.02
antmaze-large-play-v2	0.29 $\pm$ 0.05	0.29 $\pm$ 0.02	0.37 $\pm$ 0.02	0.24 $\pm$ 0.04	0.28 $\pm$ 0.05	0.0 $\pm$ 0.0
kitchen-partial-v0	0.7 $\pm$ 0.03	0.73 $\pm$ 0.06	0.7 $\pm$ 0.04	0.68 $\pm$ 0.05	0.7 $\pm$ 0.06	0.3 $\pm$ 0.06
kitchen-mixed-v0	0.47 $\pm$ 0.06	0.47 $\pm$ 0.06	0.49 $\pm$ 0.07	0.52 $\pm$ 0.06	0.58 $\pm$ 0.03	0.4 $\pm$ 0.04
door-binary-v0	0.26 $\pm$ 0.04	0.31 $\pm$ 0.19	0.24 $\pm$ 0.03	0.21 $\pm$ 0.07	0.08 $\pm$ 0.08	0.05 $\pm$ 0.04
pen-binary-v0	0.69 $\pm$ 0.03	0.66 $\pm$ 0.05	0.74 $\pm$ 0.03	0.75 $\pm$ 0.05	0.58 $\pm$ 0.03	0.89 $\pm$ 0.04
Average	0.42 $\pm$ 0.04	0.41 $\pm$ 0.04	0.45 $\pm$ 0.04	0.4 $\pm$ 0.04	0.39 $\pm$ 0.04	0.25 $\pm$ 0.05

Table 5 reports success rates (mean $\pm$ SE) at the end of the offline pretraining phase, i.e., the start of online fine-tuning, averaged over five seeds. Despite identical offline training procedures, end-of-offline performance shows small variations across ANCHOR, WSRL, PORL, Cal-QL, and CQL. These differences reflect evaluation stochasticity rather than algorithmic effects.

Table 6: **Catastrophic failure (0-400K window). Bold** indicates the lowest catastrophic failure, and underline indicates the second lowest.

Task	**ANCHOR (Ours)**	WSRL	PORL	Cal-QL	CQL	IQL
antmaze-large-diverse-v2	0.12 ± 0.07	**0.07** ± 0.04	0.39 ± 0.04	0.17 ± 0.07	0.21 ± 0.04	0.03 ± 0.02
antmaze-large-play-v2	**0.11** ± 0.09	0.2 ± 0.06	0.37 ± 0.02	0.16 ± 0.05	0.16 ± 0.07	0.0 ± 0.0
kitchen-partial-v0	**0.15** ± 0.08	0.68 ± 0.09	0.7 ± 0.04	0.68 ± 0.05	0.7 ± 0.06	0.19 ± 0.04
kitchen-mixed-v0	**0.22** ± 0.11	0.42 ± 0.09	0.48 ± 0.07	0.52 ± 0.06	0.58 ± 0.03	0.35 ± 0.04
door-binary-v0	0.26 ± 0.04	0.31 ± 0.19	0.24 ± 0.03	0.21 ± 0.07	0.08 ± 0.08	0.05 ± 0.04
pen-binary-v0	**0.14** ± 0.04	0.52 ± 0.04	0.66 ± 0.04	0.33 ± 0.08	0.58 ± 0.03	0.87 ± 0.03
Average	**0.16** ± 0.03	0.34 ± 0.05	0.44 ± 0.03	0.31 ± 0.04	0.36 ± 0.04	0.23 ± 0.05

Table 7: **Catastrophic failure (0-100K window). Bold** indicates the lowest catastrophic failure, and underline indicates the second lowest.

Task	**ANCHOR (Ours)**	WSRL	PORL	Cal-QL	CQL	IQL
antmaze-large-diverse-v2	0.1 ± 0.08	**0.07** ± 0.04	0.39 ± 0.04	0.17 ± 0.07	0.21 ± 0.04	0.03 ± 0.02
antmaze-large-play-v2	**0.11** ± 0.09	0.2 ± 0.06	0.37 ± 0.02	0.16 ± 0.05	0.16 ± 0.07	0.0 ± 0.0
kitchen-partial-v0	**0.06** ± 0.02	0.68 ± 0.09	0.7 ± 0.04	0.68 ± 0.05	0.7 ± 0.06	0.18 ± 0.04
kitchen-mixed-v0	**0.03** ± 0.01	0.42 ± 0.09	0.48 ± 0.07	0.52 ± 0.06	0.58 ± 0.03	0.33 ± 0.03
door-binary-v0	0.26 ± 0.04	0.31 ± 0.19	0.24 ± 0.03	0.21 ± 0.07	0.08 ± 0.08	0.05 ± 0.04
pen-binary-v0	**0.14** ± 0.04	0.52 ± 0.04	0.66 ± 0.04	0.33 ± 0.08	0.58 ± 0.03	0.83 ± 0.04
Average	**0.12** ± 0.02	0.34 ± 0.05	0.44 ± 0.03	0.31 ± 0.04	0.36 ± 0.04	0.22 ± 0.05

Tables 6 and 7 report catastrophic failure over the 0-400K and 0-100K windows, respectively. All values are averaged over five seeds and reported as mean ± SE. The estimates are similar across the two windows, indicating that most failures occur early in the offline-to-online transition. ANCHOR achieves the lowest average catastrophic failure. For clarity, we do not highlight IQL, whose low failure mainly reflects weak post-offline performance rather than stable adaptation, or door-binary, where all methods reach zero success during online fine-tuning.

L.2 Pretrained Policy Quality Analysis with Confidence Intervals

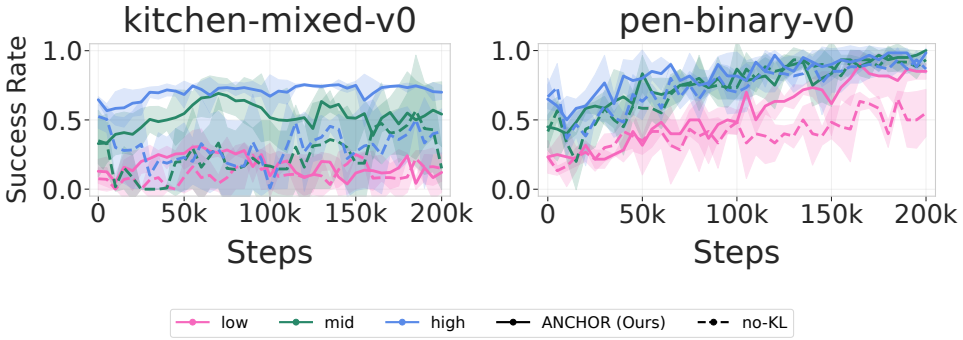


Figure 17: **Benefit of ANCHOR depends on pretrained policy quality.** ANCHOR reduces catastrophic failure more clearly when the offline-pretrained policy starts from a higher initial success rate. Curves with the same color correspond to the same pretrained policy. Results are averaged over three seeds.

Figure 17 reports the pretrained-policy-quality analysis from Section 5.3 with full confidence intervals. The results are consistent with the interpretation of Figure 6: when the pretrained policy starts from a higher initial success rate, the gap between ANCHOR and the no-KL variant becomes clearer. This shows that ANCHOR reduces catastrophic failure more effectively when there is useful pretrained behavior to preserve.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction clearly state the problem (offline-to-online RL without offline replay), our method (ANCHOR with KL anchoring and annealing), theoretical contributions, and empirical scope (FrankaKitchen, AntMaze, Adroit). All claims are supported by experiments and ablations in the paper.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Limitations are discussed in Appendix J, covering scope of evaluation (single-agent continuous control only), hand-designed annealing schedules requiring modest tuning, and limited statistical coverage due to computational constraints.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Appendix A contains one formal assumption (Assumption 1: local occupancy control around the offline policy) and complete proofs for both lemmas. The paper explicitly notes that the goal is to clarify the theoretical role of KL anchoring rather than prove a full convergence theorem, so the scope of the theoretical claims is accurately bounded.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Appendix I provides full implementation details including hyperparameters, and network architectures. Baseline reproduction protocols are also described.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: All benchmark datasets used in this paper are publicly available. We provide the code in the supplementary material, and Appendix I gives the implementation details needed to faithfully reproduce the main results.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Training and evaluation details are described in Section 5.1, with full hyperparameter tables, network architectures, and training step counts provided in Appendix I.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: All main experiments are run with five random seeds. Learning curves are shown with confidence intervals as shaded regions, and numerical results in the appendix are reported as mean $\pm$ standard error (Appendix L).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix I specifies the compute environment (8× NVIDIA A100 80GB GPUs, AMD EPYC CPU, 885GB RAM) and per-task online training time for main experiments (Table 2).

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: This work studies offline-to-online reinforcement learning on simulated robotic control benchmarks and does not involve human subjects, sensitive data, or harmful applications. The research conforms with the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: This paper proposes a foundational algorithm for offline-to-online RL adaptation and does not target a specific application with direct societal impact. No foreseeable negative societal impacts are anticipated from this work.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: This paper does not release high-risk models or datasets. The method is evaluated on simulated robotic control benchmarks, and no models with significant misuse potential are released.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All datasets and baseline codebases are properly cited in the paper. The D4RL benchmark and WSRL codebase are open-source and their usage complies with their respective licenses.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: This paper does not release new datasets or models as standalone assets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not a component of the core methods in this research.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.